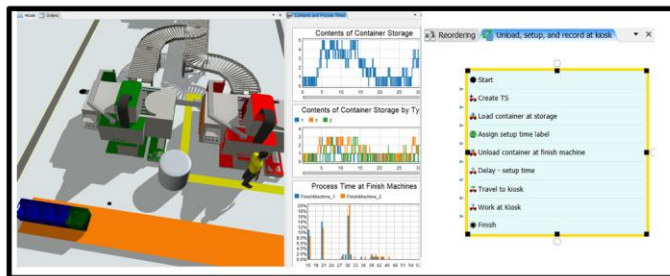
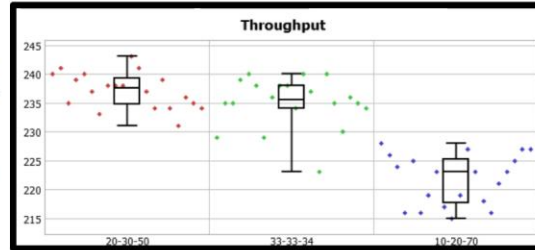
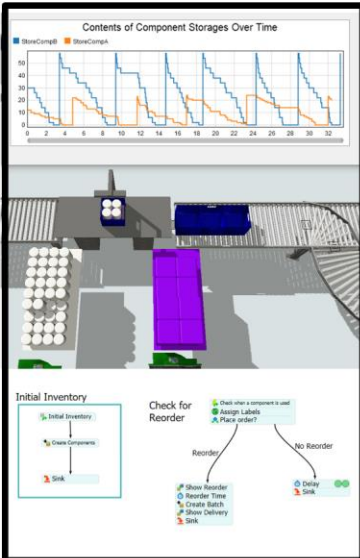
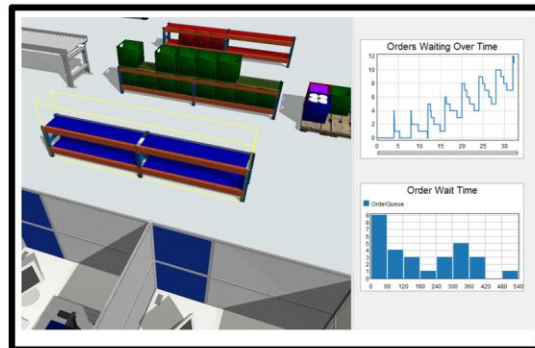




SIMULATION SOFTWARE PRIMER



Allen G Greenwood, Ph.D., P.E.

Simulation Education Specialist, FlexSim Software Products, Inc.

Professor Emeritus, Department of Industrial & Systems Engineering, Mississippi State University

Fourth Edition, 01 June 2020

Copyright © 2020 FlexSim Software Products, Inc. All rights reserved.

Published by FlexSim Software Products, Inc., Canyon Park Technology Center, Building A Suite 2300, Orem, UT 84097 USA.

FlexSim software and books may be purchased for educational, business, or sales promotional use. For more information, please contact our sales department: +1-801-224-2914 or sales@FlexSim.com.

The *FlexSim* logo is a registered trademark of FlexSim Software Products, Inc. Other registered trademarks belonging to third parties are used within this work. Where those designations appear in this book, and FlexSim Software Products, Inc. was aware of a claim, the designations have been printed in italics, caps, or initial caps.

While every precaution has been taken in the preparation of this book, the publisher and the author assume no responsibility for errors or omissions, or for damages resulting from the use of the information contained here.

Version 1.1, for software version 2017 Update 2, May 2018

Version 2.0, for software version 2018 Update 2, September 2018

Version 3.0 for software version 2019 Update 2, December 2019

Version 4.0 for software version 2020 Update 1, June 2020

Supporting files and all *FlexSim* model files are available at <https://flexs.im/primer-files>

ABSTRACT

This primer, using a single comprehensive example and sequential development process, provides detailed, step-by-step instructions for building and analyzing a comprehensive simulation model in the *FlexSim* simulation software. The system that is modeled is small, but not simple. While the focus is on basic concepts and constructs in *FlexSim*, the primer introduces some of the advanced features and capabilities that are available in *FlexSim*. The primer describes: (1) *FlexSim*'s basic fixed and mobile 3D objects (e.g., processors, conveyors, operators) that are powerful and yet easy to use, (2) the ability to build custom logic through the flowchart-like tool Process Flow, (3) other powerful tools such as tables, lists, reliability, and charts, (4) designing and running experiments with a simulation model to analyze the operational dynamics of a system, and (5) *FlexSim*'s array of modules for modeling AGVs, warehouses, people, etc. In addition to basic instructions, the primer provides insight and rationale for the described modeling actions and introduces good modeling and analysis practices.

The primer is not a software user manual that is primarily a reference document. The primer provides a means to learn the basics of *FlexSim* simulation software and to be introduced to its power and capabilities in a way that demonstrates how it relates to the simulation problem-solving process. The primer is primarily intended for someone who has little or no familiarity with *FlexSim* simulation software. However, in addition to new users, it is hoped that existing *FlexSim* users will find the primer useful to clarify some aspects of the software and to learn something new. While it is preferable that the reader is familiar with the basics of discrete-event simulation, that is not required.

Supporting files and all *FlexSim* model files are available at <https://flexsim.com/primer-files>

This page is intentionally blank.

PREFACE

This primer is based on many years of experience and my long-time passion for simulation. I started learning simulation in graduate school and applying it in practice in my first position as an industrial engineer in the mid-1970s. Simulation has been a major part of my professional life in a variety of settings and situations. These include teaching many simulation courses at the undergraduate and graduate levels at a number of universities, both in the US and other countries; developing and delivering short courses for practicing engineers in a variety of industries; writing, presenting, and publishing research papers on various aspects of simulation modeling and analysis; and, carrying out many simulation projects in a range of industries.

I consider simulation to be an applied technology that is an essential tool for effectively supporting a broad range of problem-solving and decision-making processes. It is especially valuable for the design and management of systems in many domains, such as manufacturing, material handling, warehousing, logistics, healthcare, mining, business processes, etc. All of these domains are naturally complex – the complexities are due to the systems having many disparate elements, the relationships among those elements, inherent variability, and intrinsic dynamics. Simulation facilitates and encourages the exploration and assessment of multiple ideas and alternatives in order to “optimize” system performance. All of this is done virtually without disturbing an existing system or before a system actually exists.

There are many good resources available to learn about simulation modeling and analysis in general and *FlexSim* in particular, including textbooks (such as *Applied Simulation Modeling and Analysis Using FlexSim*), training and teaching materials, user manuals, tutorials, blogs, videos, etc. Each resource has its own objectives and is devised to meet specific needs. However, none of the resources by themselves meet all of the primer’s objectives:

- provide detailed, step-by-step instructions for building a comprehensive simulation model,
- use a single comprehensive example and sequential development process to effectively move from introducing and applying simple concepts and methods to describing those that are more complex,
- provide insight and rationale for modeling actions, rather than just providing rote commands,
- introduce good modeling and analysis practices,
- introduce fundamental concepts of simulation; provide references for further reading, and
- provide an awareness of some of the more advanced features available in *FlexSim* without covering the details at that time.

This primer focuses on the modeling and analysis of operations systems in order to understand and analyze their dynamic behavior and performance (referred to as *operations dynamics*) using discrete-event simulation. In the broadest sense, operations systems transform input into output through a set of related activities and processes that require a variety of resources, such as equipment, material, people, and information. Transformations may either be tangible (e.g. machining, inspecting, or delivering material in manufacturing) or intangible (e.g. diagnosing or treating patients in healthcare).

Simulating operations dynamics requires an accurate representation of the system being considered, including both the physical aspects (locations, distances, sizes, speeds, etc.) and the salient underlying logic (e.g., what action to take, when and where to perform the action, using what resources). Visualization of the behaviors

resulting from the interaction of the physical and logical aspects greatly increases the acceptance of modeling and analysis by decision makers and those that are not experts in simulation technologies and methodologies. Visualization also enhances model validation and verification. *FlexSim* has rich visual and logical functionality and is easy to use. Of course, ease of use is relative, depending on a user's background and experience.

Since *FlexSim* is so comprehensive, it is not feasible to cover all of its capabilities in this introductory guide. Similarly, the field of simulation is extensive; therefore, the theory and methods of simulation modeling and analysis is beyond the scope of this basic primer. Therefore, for an introduction to simulation, the following primer is suggested. (The author of this primer is also author of the simulation primer.)

Greenwood, A. *Simulation Primer*, FlexSim Software Products, Inc., 2019.



For a more detailed discussion of simulation modeling and analysis concepts and practices, and how they can be implemented in *FlexSim*, the following textbook is suggested. (The author of this primer is a coauthor of the textbook.)

Beaverstock, M., Greenwood, A., and Nordgren, W. *Applied Simulation Modeling and Analysis Using FlexSim*, 5th Edition, FlexSim Software Products, Inc., 2017.



Other sources of information on *FlexSim* include: the *FlexSim User Manual*, in-software links to the manual, tutorials, and web resources, especially *FlexSim Answers*, which is a shared searchable knowledge base of questions and answers from FlexSim's worldwide community of users.

It is important to note that this primer is not another user manual for the software. User manuals are good reference resources to describe specific aspects of the software; whereas this primer is a means to learn both the basics of *FlexSim* simulation software and to be introduced to its power and capabilities in a way that demonstrates how it relates to the simulation problem-solving process.

It is suggested that this primer be covered carefully and methodically in the order it is written. The material builds from very simple aspects of simulation and modeling to the more complex. However, someone with more experience with simulation and the software may skim the early material, but it still should be considered since, at a minimum, it provides context. Later sections then may be considered in more detail to enhance understanding of a concept or software capability. Since all of the models described in the primer are available online (<https://flexsim.com/primer-files>), more experienced readers can start at a later section without having to build all of the previous models. Of course, providing all of the models also benefits those new to the software.

Allen G Greenwood

revised December 2019, Greensboro, NC USA
September 2018, Blowing Rock, NC USA

TABLE OF CONTENTS

ABSTRACT	3
PREFACE.....	5
TABLE OF CONTENTS	7
INTRODUCTION.....	11
OBJECTIVES.....	11
STRUCTURE	11
APPROACH.....	12
TESTIMONIALS.....	13
WHY <i>FLEXSIM</i> SIMULATION SOFTWARE?	13
THE ROAD TO CHOOSING <i>FLEXSIM</i>	15
FLEXSIM SIMULATION SOFTWARE	17
PART 1 – THE BASICS OF 3D MODELING AND ANALYSIS	19
1 FUNDAMENTAL CONCEPTS	20
1.1 SIMULATION MODELING AND ANALYSIS.....	20
1.2 SIMULATION USING <i>FLEXSIM</i>	21
1.3 OBJECT LIBRARY	21
1.4 TOOLBOX.....	22
1.5 ANALYSIS.....	22
1.6 NOTATION AND FORMATTING CONVENTIONS	23
2 DESCRIPTION OF THE SYSTEM TO BE MODELED.....	25
2.1 OVERVIEW.....	25
2.2 MODELING APPROACH AND BASIC SYSTEM DESCRIPTION	26
3 FLEXSIM'S MODELING ENVIRONMENT.....	28
3.1 TIME AND DISTANCE UNITS.....	29
3.2 MAIN MENU AND MAIN TOOLBAR.....	30
3.3 MODEL EXECUTION TOOLBAR.....	31
3.4 3D MODEL VIEW WINDOW, MODELING SURFACE, AND MOUSE OPERATIONS	31
3.5 OBJECT LIBRARY AND TOOLBOX	33
3.6 OBJECT INTERFACE	33
3.7 QUICK PROPERTIES PANEL.....	35

3.8	HELP.....	35
3.8.1	User Manual.....	35
3.8.2	Context Help.....	36
3.8.3	Answers.....	36
4	GETTING STARTED WITH THE SIMPLEST MODEL.....	37
5	BASIC FIXED RESOURCE OBJECTS AND CUSTOMIZATION	41
5.1	BASIC OBJECT PROPERTIES AND STRUCTURE	41
5.2	SOURCE OBJECT.....	44
5.3	FLOWITEM BIN	47
5.4	QUEUE OBJECT.....	48
5.5	PROCESSOR OBJECT	48
5.6	SINK OBJECT	50
6	BASIC MODEL OUTPUT.....	51
6.1	OBJECT STATISTICS AND QUICK PROPERTIES.....	52
6.2	DASHBOARD.....	54
	PART 2 – NEXT STEPS IN 3D MODELING.....	61
1	CHANGING 3D GRAPHICS.....	62
1.1	IMPORTING A LAYOUT	62
1.2	CHANGING OBJECT GRAPHICS	65
2	ALTERNATIVE ROUTINGS	69
2.1	CHOOSING A ROUTE BASED ON AVAILABILITY	69
2.2	CHOOSING A ROUTE BASED ON CURRENT SYSTEM CONDITIONS.....	70
3	TASK EXECUTERS.....	71
3.1	BASIC TASK EXECUTER CONCEPTS	71
3.2	ADDING A FINISH OPERATOR TO THE MODEL	72
3.3	CONTROLLING TASK EXECUTER TRAVEL WITH PATH NETWORKS	76
4	CONVEYORS.....	80
4.1	STRAIGHT AND CURVED CONVEYOR SECTIONS	81
4.2	CONNECTING CONVEYOR SECTIONS.....	83
4.3	JOIN CONVEYORS OBJECT.....	84
4.4	CONVEYOR TYPES.....	85
5	GLOBAL TABLES	87
5.1	USING TABLES TO STORE PROCESS TIMES.....	88
5.2	USING TABLES TO STORE PRODUCT MIX PERCENTAGES.....	89

6	DOWNTIME.....	91
6.1	TIME TABLES.....	91
6.2	RELIABILITY.....	95
6.3	CONSTANT MTTF/MTTR; MTBF BASED ON CLOCK TIME; NO RESOURCE FOR REPAIR	95
6.4	RANDOM MTTF/MTTR; MTBF BASED ON SYSTEM STATES; RESOURCE FOR REPAIR	97
6.5	CHARTING THE EFFECT OF DOWNTIME.....	99
PART 3 – MORE 3D MODELING + ANALYSIS	103	
1	COMBINING AND SEPARATING FLOWITEMS	104
1.1	SCHEDULED ARRIVALS OF COMPONENTS	104
1.2	MODELING THE PACKING OPERATION USING THE COMBINER OBJECT.....	108
1.3	DOWNTIME AT THE PACKING OPERATION	113
1.4	BATCHES OF COMPONENTS UNLOADED BY THE OPERATOR USING SEPARATOR OBJECTS	115
2	CONTROLLING TASK EXECUTER TRAVEL PATHS USING A*.....	123
2.1	THE BASICS OF THE A* ALGORITHM USING A SIMPLE STUDY MODEL.....	123
2.2	IMPLEMENTING THE A* ALGORITHM IN THE PRIMER MODEL	127
2.3	SAVING MODEL VIEWS.....	131
3	USING AN ITEM LIST FOR COMPLEXMODELING LOGIC	132
3.1	SIMPLE PRIORITY ROUTING USING AN OBJECT TRIGGER.....	132
3.2	MULTIPLE-CRITERIA ROUTING USING LISTS	133
4	EXPRIMENTATION IN <i>FLEXSIM</i>.....	139
4.1	EFFECT OF BUFFER SIZE ON PERFORMANCE.....	139
4.2	EFFECT OF PRODUCT MIX ON PERFORMANCE	143
5	SUMMARY OF THE PRIMER FOR <i>FLEXSIM</i> 3D	147
PART 4 – INTRODUCTION TO PROCESS FLOW	149	
1	BASIC CONCEPTS.....	150
1.1	BASIC PROCESS FLOW CONCEPTS AND MODELING ENVIRONMENT	150
1.2	USING PROCESS FLOW TO MODEL INVENTORY POLICY	152
2	MODELING INITIAL INVENTORY	153
3	MODELING INVENTORY REORDER POLICY	158
3.1	CHANGES IN 3D OBJECTS FOR USE IN PROCESS FLOW	158
3.2	IMPLEMENTING REORDER-POINT INVENTORY LOGIC USING PROCESS FLOW.....	160
3.3.1	“CHECK FOR REORDER” LOGIC.....	160

3.3.2	“REORDER” LOGIC	162
3.3.3	“NO REORDER” LOGIC.....	164
4	CUSTOM TASK SEQUENCES IN PROCESS FLOW	166
4.1	BASE MODEL MODIFICATIONS	166
4.2	SIMPLE STUDY MODEL OF TASK SEQUENCES IN PROCESS FLOW	168
4.3	CUSTOM TASK SEQUENCES IN THE PRIMER MODEL.....	174
	PART 5 – WAREHOUSING, ORDER PROCESS AND ADDITIONAL FEATURES	181
1.	INTRODUCTION TO WAREHOUSE MODULE.....	182
2.	ORDER PROCESSING SUBMODEL	185
2.1	DEFINITION OF THE ORDER-FULFILLMENT PROCESS	185
2.2	IMPLEMENTATION OF THE ORDER-FULFILLMENT PROCESS	185
3	OTHER FLEXSIM OBJECTS AND CAPABILITIES	194
3.1	MULTIPROCESSOR OBJECT	194
3.2	TRANSPORTER OBJECT.....	195
3.3	ROBOT AND CRANE OBJECTS	195
3.4	FLUIDS LIBRARY	196
4	SUMMARY OF THE PRIMER MODEL	197
	EPILOGUE	200
	APPENDIX - LIST OF MODELS.....	202
	APPENDIX - GLOSSARY	207
	ABOUT THE AUTHOR	214

Supporting files and all *FlexSim* model files are available at <https://flexsim.com/primer-files>

INTRODUCTION

This section provides the primer's objectives, organization, and general approach.

Objectives

As stated in the Preface, the objectives of this primer are to:

- provide detailed, step-by-step instructions for building a comprehensive simulation model in *FlexSim*,
- use a single comprehensive example and sequential development process to effectively move from introducing and applying simple concepts and methods to describing those that are more complex,
- provide insight and rationale for modeling actions, rather than instructing rote commands,
- introduce good modeling and analysis practices,
- introduce fundamental concepts of simulation; provide references for further reading, and
- provide an awareness of some of the more advanced features available in *FlexSim* without covering the details at that time.

Structure

In order to meet these objectives, the primer is divided into the following five parts:

1. *The Basics of 3D Modeling and Analysis* that include a brief introduction to the following: fundamental simulation concepts, description of the system to be modeled, overview of *FlexSim*'s modeling environment, how to build a very simple model, fundamentals of *FlexSim*'s fixed resource objects and customization, and introduction to output from *FlexSim* models.
2. *Next Steps in 3D Modeling* continues through the following topics: changing 3D graphics, incorporating alternative routing rules, introduction to Task Executors or dynamic objects, basics of modeling conveyors, the use of Global Tables for data management, and incorporating planned and unplanned downtime into *FlexSim* models.
3. *More 3D Modeling and Analysis* discusses the following topics: combining and separating flowitems, controlling Task Executors using the A* algorithm, using Lists to represent complex routing logic, and running experiments in *FlexSim*.
4. *Introduction to Process Flow* begins with a discussion of the basic concepts of the logic builder and then provides two examples of building custom logic – one for creating inter-object logic to implement a reorder-point inventory policy and the second for creating a custom task sequence for a mobile resource (Task Executor).
5. *Order Fulfillment Sub-Model, Warehousing, and Additional FlexSim Features* introduces *FlexSim*'s Warehousing Module and adds logic to the primer model that processes incoming orders and completed items from production. This part also briefly introduces some of additional capabilities in *FlexSim*, including other 3D objects and the Fluids Library.

The main portion of the primer concludes with an *Epilogue* that provides a brief summary and possible next steps.

The primer also includes two Appendixes – a *List of Models* and a *Glossary*.

- The list of models provides a brief description of the concepts and software features that are used in each of the primer models, as well as a reference to the section in the primer where the model is discussed.
- The glossary provides definitions of key terms used in the primer.

The primer concludes with a brief biography of the author and his contact information.

In the following section, prior to the main parts of the primer, two common questions are addressed that often arise from those who are unfamiliar with *FlexSim* simulation software.

- Why should I use *FlexSim*? In order to help address this question, two long-time users of *FlexSim* provide testimonials as to why *FlexSim* is their simulation software of choice and their path to choosing *FlexSim*. One testimonial is from an academic perspective and the other is from an industry perspective.
- What is *FlexSim* and what are its basic capabilities? In order to help address this question, a brief overview of the software is provided along with information on how to acquire the software.

Approach

As mentioned in the objectives above, the primer introduces and explores *FlexSim* through a single model. The model is small, but not simple. Its complexity evolves throughout the primer as various aspects of the simulation software are presented.

Since the model of the example system evolves throughout the primer, it is recommended that each iteration of the model be saved. After each save, it is suggested that the current model file be copied and renamed so that one can always revert to a previous version of the model. This is just good modeling practice – models should always be developed in steps – build, test, and validate – and all models should start simple and then complexity is added as needed. That is, the final model evolves from a number of simpler models with increasing representation of the real system. One should never try to incorporate all capabilities at the beginning of the modeling process.

In order to facilitate learning, all of the models developed in the primer are available so that if a reader has a problem developing any one of the primer models, they can access and view the correct implementation and then proceed on with the primer. Having access to the primer models also allows a reader with some knowledge of *FlexSim* to skip the simpler parts of the model building process and start at any section of the primer. For example, if a reader is well versed in the 3D aspects of *FlexSim*, but is not familiar with Process Flow, then they can start the primer in Part 4 and begin with the model from the end of Part 3.

The models, along with supporting files (a layout file and two 3D representations of work areas in the example) are provided in a resources folder that is available at <https://flexs.im/primer-files>.

TESTIMONIALS

This section provides two testimonials, one from an academic perspective and one from an industry perspective, that discuss why *FlexSim* is their simulation software of choice and their path to choosing *FlexSim*. Each testimonial is from a long-time user of *FlexSim* and co-author of the textbook, *Applied Simulation Modeling and Analysis Using FlexSim*.

Why *FlexSim* simulation software?

Provided by: Allen G Greenwood, Ph.D., P.E.
Professor Emeritus, Mississippi State University
Simulation Education Specialist, FlexSim Software Products, Inc.

There are a number of simulation software products available in the marketplace; some have been around for many years, while others are quite new. A common question, both in industry and academe, is why should *FlexSim* be used, instead of any of the other software that are available?

I believe that the choice of a software tool should align with a person's general approach to modeling and simulation. Therefore, I briefly introduce my background in order to give context and perspective to my response to the posed question. I have been involved with simulation for over 40 years – using it to solve problems in a variety of industries and teaching numerous simulation courses, both in the U.S. and abroad. Over the years I have used numerous simulation software products in industry projects, to support research, and in the classroom. I started using *FlexSim* in 2006 for industry projects and then, a few years later, transitioned my simulation courses to use *FlexSim*. Since then, I have co-authored the textbook, *Applied Simulation Modeling and Analysis using FlexSim* (now in its fifth edition) and recently authored two primers, one focused on using *FlexSim* software and one on simulation in general. One of my long-term, foundational professional goals has been to enhance and increase the application of simulation, both in practice and in academe, to support problem solving and decision making.

While there are many good simulation products available in the marketplace, I find *FlexSim* to be the best for my approach to solving problems and for educating others. Hopefully this essay will articulate the reasons why *FlexSim* became my simulation software of choice.

FlexSim is a modern, comprehensive simulation modeling and analysis environment that supports problem solving and decision making in industry, as well as supports teaching and research in academe. However, it was *FlexSim* - more than any other software - that made it possible for me to focus on solving industrial problems or on teaching simulation concepts. It is a software tool that is extremely powerful, yet easy to use and easy to transition, as needed, from small, simple models to those that are large and complex. This is important in practice for incremental model building – starting simple and adding complexity as needed – as it allows all those involved in the simulation process to follow and understand the underlying approach. It is especially important in teaching as I am able to have students quickly build simple models that illustrate key concepts, and then easily add desired enhancements. *Flexsim* makes it possible for me, as well as for students, to

concentrate on solving problems or learning simulation concepts, rather than dealing with the idiosyncrasies and learning curve associated with software.

Being involved in various industries and trying to give my students a basic understanding of the applicability and power of simulation, I find that *FlexSim* can easily model a wide range of operational systems including, but not limited to, discrete-part and continuous (e.g. oil and gas, food processing) manufacturing, transportation, logistics, healthcare, mining, construction, business and service processes, etc. *FlexSim* also offers a variety of simulation modeling approaches. While the core technology is discrete-event simulation, continuous and hybrid systems can be modeled through a library of pre-defined, yet customizable, fluid objects. Agent-based simulation and Monte Carlo simulation can also be performed within *FlexSim*.

Standard discrete-event and continuous modeling functionality are implemented through easy-to-use 3D objects. Most importantly, they can be customized and have common and intuitive user interfaces. The processing logic and behavior, as well as the appearance, of the objects are modified through extensive drop-down menu lists and direct parameter specifications. An intuitive, flowchart-like logic builder is available for defining and refining inter-object and intra-object relationships and behaviors. While some software claim they do not require the use of detailed computer code, there are times when it is necessary. Many of the simulation projects that I have been involved with during my career required representing complex logic and behaviors. I found that, when needed, *FlexSim*'s process logic builder (Process Flow) and inherent scripting language *Flexscript* (a subset of C++) are effective and convenient means for representing complex operations. Numerous commands and templates, i.e. prebuilt sets of code and logic, are available to facilitate customization and reduce the amount of programming that is required. *FlexSim*'s open, object-oriented, hierarchical software architecture also makes it a powerful and effective modeling environment.

FlexSim models are built in a native 3D environment. For me, their effective animation facilitates model validation, enhances stakeholder communication and understanding, and increases confidence in modeling and analysis. From a student perspective, the models look realistic, which provides needed relevance and helps make simulation exciting. In addition to the objects, I regularly use the many powerful modeling tools that are available in *FlexSim*, such as dashboards, time tables for managing work schedules, reliability tables for managing downtimes, global data tables, global variables, lists, macros, animation editor, video and model flythrough support, etc.

The simulation of an operation is only as good as the model's representation of the system and the analyses that are performed using the model. I believe this is important fundamental for students to realize. One of the reasons that I like *FlexSim* is that it provides a comprehensive and easy-to-use set of analysis tools, such as an experimenter for running multiple scenarios and replications simultaneously in a designed experiment (while taking advantage of the number of cores on your computer), statistical measures of performance, extensive plotting and charting capabilities, etc.

Simulation is only one tool for problem solving; and thus, communication with other supporting software is critical. For example, *FlexSim* provides convenient means to transfer model data both to and from spreadsheets and databases. *ExpertFit* is available to support modeling randomness (selecting probability distributions). Also, *FlexSim* provides a seamless interface to *OptQuest* for intelligently searching for the best solution, i.e. the best set of operating conditions for the system being modeled.

Another very important characteristic of simulation software is the company behind the product. FlexSim Software Products, Inc. was founded in 1993 and the first version of the *FlexSim* software was released in 2003. Customer support – for both industrial and educational users – is excellent. They truly want to help everyone effectively solve problems through the use of simulation. The company continuously improves their software through new and improved modeling and analysis capabilities, more effective interfaces and methods, better graphics, etc. FlexSim strongly supports education through student and education licenses and a variety of learning materials.

I hope this essay provides a start for understanding why I use *FlexSim* and perhaps why you should use *FlexSim* to enhance problem solving and decision making, provide an effective foundation for teaching simulation, and support research. Of course, the best way to be convinced that *FlexSim* is the software for simulation is to give it a try. Therefore, visit flexsim.com and contact FlexSim about how to get started simulating using *FlexSim*.

The road to choosing *FlexSim*

Provided by: Mal Beaverstock, Ph.D.
Manger of Business Simulation, General Mills (retired 2008)

Over a span of 12 years, more than \$150 million in savings were attributed to simulation at General Mills, Inc. (GMI). Now retired, I look back and realize that it was achieved through maintaining a problem-solving approach to simulation and to the tool that made it possible – *FlexSim*.

Early in my career, I lost faith with modeling and simulation. Attempts to use this technology to solve problems met with frustration and little results. Computing technology in the late 1960's made detailed mathematical models expensive, cumbersome, time consuming, and widely inaccurate. Problems were actually solved sooner by the involved engineers before a modeling approach could gain any momentum.

The change started in the early 1990's when a co-worker at International Paper solved a problem by modeling the mechanics of a machine aided by software that greatly facilitated building differential equations. The speed and ease of using such a tool re-ignited my interest in modeling and simulation.

While managing a control and simulation group at General Mills in 1995, an *Extend* simulation showed a savings of over \$1M by eliminating a packing line in a new design. That got management's attention! However, the work was still slow going. Using simulation software meant spending more time working with their idiosyncrasies than thinking about the problem that had to be solved.

Despite the drawbacks, successful simulations still helped by: increasing the productivity of a complex conveyor system; resolving design issues when sizing surge and storage tanks; and improving a batch operation to meet new production levels without any capital expense. However, the use of simulation remained a specialty for a few individuals and not a general tool.

A search began for a software tool that: could be accepted and used by all levels of individuals interested in simulation; provided for easy communication of assumptions and results; allowed a focus on the process dynamics; was flexible enough to handle complex functionality; and was a fully supported commercial product.

Simulations were developed using a variety of software including: *Extend*, *Witness*, *ProModel*, *Arena*, and *Excel*. None matched the requirements.

In 2002 General Mills acquired Pillsbury where *Taylor ED* was used for simulation and through their contacts, I learned about *FlexSim*. This newly-developed software showed great potential. It was based on their incorporating object-oriented software constructs – an approach I immediately recognized from my work some years earlier as head of Systems Research at the Foxboro Company. Their use of functional objects combined with state-of-the-art graphics was exceptional. It was the first time I saw such techniques applied to building a simulation tool and it set them apart from the rest of the field.

Bill Nordgren, President of FlexSim Software Products, Inc., had a similar vision to mine for a simulation tool. The resulting symbiotic relationship resulted in a product that met our functional requirements. A graduate student majoring in art at the University of Minnesota spent a summer creating 3D objects based on actual pieces of manufacturing equipment as well as objects representing General Mills products. The object interfaces were modified to reflect GMI terminology and contain design information about equipment such as standard speed, costs, and reliability information. Stored in libraries, equipment could be inserted into a simulation model without any other programming. While looking like a custom tool, the simulation program was actually a library within standard *FlexSim* and could be changed back with a single click. Since *FlexSim* served as the functional base, complex logic and dynamic simulations were possible.

As a result of *FlexSim*, by 2004 all interested individuals, including engineers, managers, and production personnel on the plant floor, could interact and contribute to the building, running, and analysis of the simulations that were helping them. Because of the visualization, people could see **their** equipment making **their** products.

The interest in simulation increased exponentially. Simulations were built 30% faster with *FlexSim*. Capital projects using simulation came in on time and under budget. *FlexSim's* flexibility allowed for simulations to: quickly provide a feasibility estimate for new manufacturing concepts; help in the design of new equipment by analyzing operator actions; optimize material prep operations; study personnel assignments and requirements; determine yogurt scheduling approaches; resolve problems in delivery and shipping areas; review scheduling impacts on entire production lines, and many others. When more complex logic was required, the consistent *FlexSim* environment made the simulation effort easier to develop and understand.

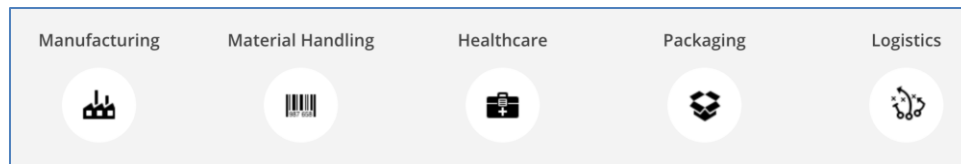
While the innovative software design, development execution, and tool-like approach to *FlexSim* was a major factor in my selection, the vision, responsiveness and commitment of the entire FlexSim staff sealed the deal. As far as I was concerned, the decision to use *FlexSim* was a “no-brainer” and proved itself in practice.



FLEXSIM SIMULATION SOFTWARE

FlexSim is a powerful, yet easy to use, environment for developing and analyzing simulation models of complex operations systems. Some of its features and capabilities include:

- comprehensive libraries of modeling objects and tools,
- modeling directly in 3D,
- complementary means to develop simulation models, such as customization of standard objects via consistent intuitive interfaces, developing complex process logic through a flowchart-like logic builder, and specifying behavior through custom coding with a scripting language that is a subset of C++,
- open, object-oriented, hierarchical architecture,
- links to spreadsheet, database, statistical, optimization, and graphics software,
- experimenter for designing, executing, and analyzing multiple scenarios and direct connection to *OptQuest* for optimization, and
- general modeling environment to support problem solving and decision making in diverse domains and applications.



- ...

This primer is based on the *Enterprise* or *Education* versions of *FlexSim*, each of which requires a license. While *FlexSim Express* is available for free download from flexsim.com, it is primarily for evaluation purposes; and as a result, it is limited in the number of objects that can be used in a model and lacks key features of *FlexSim*, such as full customization and the Experimenter.

While FlexSim Software Products provides access to all versions of its software on its website, flexsim.com, it promotes two versions of *FlexSim* at any time - Current and LTS (Long-Term Support). At the time of writing this primer, Current is version 20.1.2 (release date 2020-05-08) and LTS is version 20.0.6 (release date 2020-05-08). A major release occurs at the end of the previous year, e.g. version 2020 (20.0.0) was released in late December 2019. Two updates with new features and capabilities occur twice a year, Update 1 is released in the March/April timeframe (e.g., 20.1.0) and Update 2 is typically released in the July/August timeframe (e.g., 20.2.0).

The current version of the textbook *Applied Simulation: Modeling and Analysis Using FlexSim* 5th Ed. (2017) is based on the version 2017.0.13.

This primer is based on the Current version - 2020 Update 1 (20.1.2, release date 2019-05-08). While many of the basic operations of the software and user interfaces are the same or very similar, there may be differences with other versions.

To obtain this version, log into your FlexSim account. The following dashboard will display once logged in.

FlexSim
problem solved.

Dashboard Downloads Licenses Support Allen

Product Roadmap Announcement
Starting with the launch of FlexSim 2021, new FlexSim releases will be offered in 64-bit only. [Learn more.](#)

Welcome to FlexSim
[Download](#) the latest version of the software.
View the [FlexSim Basics Tutorial](#)
Contact your local distributor for a live Web Demo to see FlexSim in action.

Questions? Need help?
For questions about the software, follow these links or visit the [support](#) page for more information.
For training courses visit your distributor's profile

🔍 FLEXSIM ANSWERS 📖 FLEXSIM DOCS

Try, Buy, Renew?
Want to try a full version of FlexSim, get a purchase quote, or renew your existing licenses? Contact your local distributor.
To view your active licenses, visit the [licenses](#) page.

Local FlexSim Distributor

FlexSim Software Products, Inc.

Canyon Park Technology Center
1577 North Technology Way
Building A, Suite 2300
Orem, Utah 84097 USA
(801) 224-6914
sales@flexsim.com
www.flexsim.com
[View Profile](#)

Select **Downloads** from the main menu, resulting in the following interface.

FlexSim
problem solved.

Dashboard Downloads Licenses Support Allen

Downloads

FlexSim 2020 Update 1
Current
Version 20.1.2 Release Date: 2020-05-08
[Release Notes](#)
[System Requirements](#)
[Download](#) installer 892 MB
[Is there a user manual?](#)
[More Versions](#)

FlexSim 2020
LTS [Long Term Support](#)
Version 20.0.6 Release Date: 2020-05-08
[Release Notes](#)
[System Requirements](#)
[Download](#) installer 912 MB

Product Roadmap Announcement
Starting with the launch of FlexSim 2021, new FlexSim releases will be offered in 64-bit only. [Learn more.](#)

[Software](#) [Modules](#) [FlexSim Models](#) [Educator Materials](#) [3rd Party Modules](#) [Links](#)

FlexSim 2020 Update 1
Version 20.1.2 Release Date: 2020-05-08
[Release Notes](#)
[System Requirements](#)
Powerful computer software for modeling, analyzing, visualizing, and optimizing any imaginable process.

FlexSim_20.1.2_installer.exe	892 MB	installer	64bit	Download
FlexSim_20.1.2_x64.msi	891 MB	msi	64bit	Download
FlexSim_20.1.2_x86.msi	872 MB	msi	32bit	Download

[More Versions](#)

To download the current version, select Downloads; for earlier, then click on **More Versions** (located just below the Download button for the Current version). The installer for all previous versions of *FlexSim* are available for download.

PART 1 – THE BASICS OF 3D MODELING AND ANALYSIS

The first part of the primer introduces:

- fundamental simulation modeling and analysis concepts,
- notation and format used in this primer,
- *FlexSim's*
 - overall structure
 - general modeling environment
 - basic fixed resource modeling objects
 - simple output from a simulation model
- description of the system being modeled. The same system is used throughout the primer and the model of that system evolves from a very simplistic representation of the system to one that more closely represents how the actual system might behave.

1 FUNDAMENTAL CONCEPTS

This primer begins with a short introduction to simulation modeling and analysis, an overview of *FlexSim*'s general approach to simulation, and the notation and formatting conventions used in this document.

1.1 Simulation modeling and analysis

Simulation is used for analyzing and solving problems and to support decision making. It is used to:

- understand a system's behavior, especially its dynamics,
- analyze and predict a system's performance,
- compare alternatives for improvement, and
- make the best decision for change.

Simulation is composed of two key parts, modeling and analysis:

- Simulation **modeling** is the ways and means for representing a system physically and logically in order to understand its behavior over space and time and to assess possible consequences of actions, virtually.
- Simulation **analysis** is the ways and means of using a simulation model to experiment with, and test, ideas and alternatives before deciding actions and committing resources.

While many things can be simulated, the focus of this primer, and the focus of *FlexSim*, is the simulation of operations systems – systems that transform input into output through a set of related activities and processes requiring a variety of resources, such as equipment, material, people, and information. Transformations may either be tangible (machining, inspecting, or delivering material in manufacturing) or intangible (e.g. diagnosing or treating patients in healthcare).

In order to simulate **operations systems**, three key aspects must be addressed: **interactions**, **variability**, and **dynamics** since these are all inherent in operations systems. There are complex relationships among a system's resources (material, equipment, people, and information) and the resources interact in numerous and complex ways. In even the simplest system, there are many sources of variability – arrival times and rates, process times, product mix, downtime, and quality level – to name a few. Some sources of variability are known, such as work schedules and possibly product mix (at least in the short term). Other types of variability are unknown, such as quality, process times, and breakdowns. Due to variability the resource interactions change over time, thus resulting in the system's dynamics.

A simulation must represent the basic actions that occur in an operations system, e.g. process, store, and transport items. The representation must consider physical aspects – e.g., size, distance, speed – and logical aspects – what, who, when, and where things are done, as well as how much and how long.

There are four basic types of simulation that are used to model and analyze operations systems: discrete-event simulation, Monte Carlo simulation, continuous simulation, and agent-based simulation. While all four of these types of simulation can be done using *FlexSim*, the most common, by far, is **discrete-event simulation** (DES). In DES the states of a system change at discrete points in time as the result of specific events, such as a customer arriving to a system or a work shift ending. A system **state** is a condition of a system or value of a system variable, such as whether a resource is busy or idle or how many customers are waiting for a service.

1.2 Simulation using *FlexSim*

FlexSim is a comprehensive simulation modeling and analysis environment and has extensive capabilities. Some, but not all, of the features and operations are noted below. Key terms are highlighted in bolded font.

1.3 Object Library

A library of modeling **objects** is available to rapidly represent key aspects of operations systems and create system dynamics. The objects are:

- pre-built, yet customizable, representations of actions commonly found in operations systems, for example:
 - planned and unplanned delays, such as process times, repair times, and wait for resource,
 - transportation by means of both fixed objects (e.g., conveyors or robots) and mobile objects (e.g., operators, fork trucks, cranes, and AGVs),
 - resource availability and reliability, and
 - combining and separating items.
 - ...
- dragged from a library, dropped into a 3D modeling view, and connected to create a representation of the operation of a system.
- customized to represent the characteristics of the system being studied by changing an object's **properties** or parameters.
 - The extensive set of object properties consider both **physical** aspects (size, location, capacity, speed, etc.) and **logical** aspects (processing and routing rules, sequence of activities, availability, etc.), all of which either depend upon or influence the current state of a system.
 - In addition to the many standard properties that are available in objects, user-defined properties (called **labels**) can be defined, used, and/or updated anywhere and at any time during a simulation.
 - Each object has a consistent and friendly **user interface**. Object properties are conveniently grouped by tabs on the interface. Many tabs are the same across all types of objects, thus making learning the software much easier. Properties are specified through direct value entry and drop-down menu options.
 - Properties may also be customized and controlled through *FlexSim*'s built-in logic builder, called **ProcessFlow**, or by writing computer code using *FlexSim*'s built-in scripting language, **FlexScript**, a subset of C++.
- inherently 3D; thus, simulation models are created in and run in a three-dimensional environment.
 - Any object's 3D shape can easily be customized. For example, 3D shapes created in *AC3D*, *3ds Max*, *SketchUp*, etc. can be directly imported into *FlexSim*, as can objects from the extensive, online, open-source *3D Warehouse*.
 - Capabilities to “fly through” a model and to create videos of a model in action are built into the software.

1.4 Toolbox

The **Toolbox** is a library of tools that supports modeling building through, but not limited to:

- **Global Tables** store data and easily provide input to a model or obtain output from a model.
- **Lists** dynamically store and process information during a simulation.
- **Time Tables** specify deterministic resource availability, such as shift schedules, break times, periodic inspections, etc.
- **MTBF MTTR** provide means to specify downtime by:
 - randomly making resources unavailable for a duration that is also oftentimes a random variable,
 - basing downtime on model states (e.g. processing and traveling) rather than clock time, and
 - managing multiple or competing downtimes on the same object.
- **Groups** denote objects that are similar or are acted upon in a similar manner.
- **Process Flow** builds complex inter-object and intra-object logic.
- **Dashboards** display model dynamics through charts, e.g., pie, histogram, time series.
- **Excel Import/Export** provides direct links between *FlexSim* and *MSExcel* for inputting and outputting model data.
- Other types of tools for: creating special events and actions, tracking and collecting information on a model variable, creating global variables and macros, developing a video of a model running, etc.

1.5 Analysis

Simulation models are built in order to perform analyses. *FlexSim* provides many tools for analytics, including:

- statistics tracking for:
 - automatically tracking many common measures of performance, such as throughput, content, utilization, etc.
 - defining and tracking custom, system-specific measures.
 - charting to observe system performance on a dashboard as a simulation runs or to export results for reports and presentations.
- experimentation and optimization.
 - easy-to-use, built-in **Experimenter** for creating and comparing changes to sets of model parameters (**scenarios**), running multiple **replications** of a model, providing confidence interval estimates and other types of statistics, etc.
 - direct interface to a leading **optimization** engine, *OptQuest*, in order to effectively search for the “best” set of model parameters to meet specified system objectives.
- export to analysis software – data can be exported to spreadsheets, databases, and graphing and special analysis software for further investigation.
- data import – simulation models in *FlexSim* can be driven from:
 - data stored in spreadsheets and databases.
 - probability distributions selected by using Averil Law’s *ExpertFit* software. (*ExpertFit* is included with a *FlexSim* license.)

All model data are stored in an open, accessible hierarchical data structure referred to as the **Tree**.

Many of these features will at least be introduced in this primer. However, a full discussion of them is well beyond the scope of this introductory document.

The basic operation of *FlexSim* involves the creation and execution of **events** that are based on the logic specified in a model. The events generate actions and activities that occur over time. As a result of the events occurring and/or the current state of one or more objects, items move or flow from object to object. In *FlexSim*, the items that flow through a model are called **flowitems**. The items typically move between resources, either **fixed resources** (e.g., machines, conveyors, and storage areas) or mobile resources (e.g. operators, trucks, and AGVs). Items move into and out of objects via **ports** and contain user-defined characteristics, called **labels**. Mobile resources in *FlexSim* are called **task executors** since they execute a sequence of tasks such as travel, load, unload, etc.

Over the duration of a simulation, information on the conditions (or states) of a system are gathered, summarized, and used for analysis. Typically the summary information, such as average state values (average utilization, average number waiting for service, etc.), are used to compare the performance of alternative systems.

1.6 Notation and formatting conventions

In order to enhance the readability of this primer, certain formatting conventions and notation are used throughout the document.

As introduced above, *FlexSim* includes many objects and tools to facilitate simulation model building and analysis. When referring to these entities, for readability their names are capitalized, e.g., objects such as Processor and Source and tools such as Dashboard and Time Table. This is consistent with the way these entities are identified on the *FlexSim* user interface.

Key words are bolded the first time they are introduced.

A significant part of modeling is customizing general objects, such as a Source or a Processor, to represent the system being considered. In *FlexSim* this involves specifying properties or parameters on the objects through a user interface, which primarily involves selecting from drop-down menus or directly specifying values. Therefore, for readability:

- Names of the properties or parameters, such as **Process Time**, **Send To Port**, and **OnEntry** are denoted in bolded font.
- The values of the properties, e.g. **1000.0**, **First Available**, **item.Type**, are also bolded.
- Selection options within a drop-down menu, such as **Statistical Distribution**, **First Available**, and **Data > Set Label**, are shown in bolded font.
- Names of object tabs, such as General and Flow, are not bolded.
- Instructions to the reader for carrying out specific modeling actions are highlighted by separating them from the text and preceding them by the ➤ symbol.

In addition to providing step-by-step instructions on building example models, this primer provides general tips on modeling and analysis and on effective ways to use *FlexSim*. These are denoted by the following symbols, presented in a smaller font, and indented from the main text.



General modeling tips or best practices are denoted by the M symbol.



General analysis tips or best practices are denoted by the A symbol.



FlexSim tips are denoted by the FlexSim logo.



[section or chapter reference] denotes where a topic is discussed in Beaverstock, Greenwood, & Nordgren, *Applied Simulation Modeling and Analysis Using FlexSim*, 5th Edition, 2017.



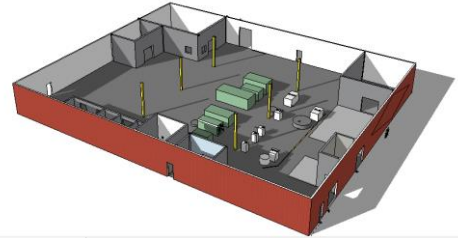
Suggested time to save a model.

2 DESCRIPTION OF THE SYSTEM TO BE MODELED

This primer uses a single model to introduce the various features and capabilities of *FlexSim*. and to provide step-by-step instructions for building a simple, yet comprehensive, simulation model.

2.1 Overview

Dobre Products Limited (DPL) is planning to reuse an area in one of its production facilities to finish and pack containers for distribution. The existing facility is shown to the right, but all of the equipment will be moved out so that the space can be used to support the new production area.



The new production area will finish various types of containers and then the containers will be packed, where the contents depend on the type of container. All process times – in both finishing and packing depend on the container type. Containers are expected to enter and leave the production area on conveyors.

At this stage of the design, DPL is not sure how many different types of containers they will produce, the demand for each type, and the components that will be packed into each container. However, they want to optimize operations and use simulation to help them design the facility and operations. Therefore, the simulation model will initially be built so that it reflects the general operations, yet contains numerous assumptions that will be modified as the project evolves. This approach is essential so that the project can be done in a timely manner - the simulation work must start well in advance of when all of the information is known. In fact, the simulation will be used to help make some of the design decisions and establish the system's characteristics and capabilities.



This is a very wise approach – *simulation is most effective when applied early in the design process* since things can easily be changed early in a project, before too many resources are committed, too many costs are incurred, and too many decisions are made.

The modeling process begins by identifying and representing the basic components of production in the finishing and packing areas (equipment, material, operators, transporters, etc.) and defining the relationships among the components. Later this is expanded, or scaled up, so that the model aligns with the company's production plans.



Modeling should be done sequentially – start small and simple and add complexity as needed. The best model is not the one that is the most complex – *the best model is the one that has the minimum amount of detail needed to answer the questions being addressed*. Remember, no model can replicate the real system – the real system is too complex. Any model is a representation, albeit a simplification, of the system being considered.

2.2 Modeling approach and basic system description

As with any simulation project, the modeling and analysis of the aforementioned system is done in steps, moving from the simplest representation to the more complex. After each step, it is important to test and validate the model. The validated model is saved in a file with a different name so that if a subsequent modeling effort encounters a problem, modeling can “rollback” to the last saved and validated version.



One of the most difficult aspects of modeling is deciding how much detail should be incorporated into a model. This is oftentimes referred to as **model fidelity**. It is very tempting to model as much detail as possible, but in this case more is not better. The more complex a model becomes, the harder it is to validate, verify, and maintain. *Models should be as detailed as needed in order to answer the posed questions.*

Of course, this is easier said than done; determining the right fidelity comes with practice and experience. In any case, modeling should start simple and complexity should be added as needed. Start modeling with an extensive list of assumptions and a very simple model. Then, iteratively decide which assumptions need to be removed and remove them by adding features to the model that address the assumptions.

Be careful not to fall into the trap where a model incorporates features and detail because it *can* be done and not because it *should* be done.



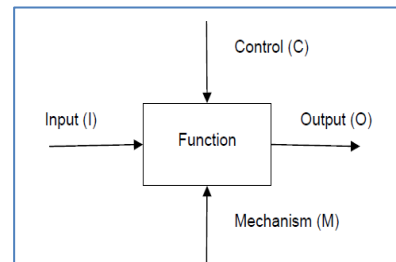
Another good modeling practice is to look for similarities within the system and model them as a single instance; i.e., *create a template*. Test and validate the instance. Once validated, copy and paste the template as many times as needed. For example, if a system has multiple packing stations, model, test, and validate one station, then duplicate as many as needed in the model.



Another good modeling practice is to use small *study models* to develop and test concepts before implementation in the main model; i.e., develop and test in isolation, removed from the complexity of the main model. This is applicable to developing logic within a single object or a set of objects.

A good approach to modeling is to *sketch* the system first. Focus on how the system *operates* and not on how the model will be *built*. Use the sketch to discuss the system operations with domain experts. One diagrammatic methodology for representing dynamic operations systems is the Object Flow Diagram (OFD).

An OFD uses symbols to represent various functions found in a simulation model, such as process, transport, convey, etc. and the edge of each symbol has a specific meaning, as shown in the figure to the right. The Input, Output, Control, and Mechanism notation is derived from the IDEF0 methodology. An OFD for the example model that is used in this primer is provided in the next section.



[Section 17.3] Prepare a conceptual model.

The following is a very brief description of the system that will be modeled throughout this primer. It is referred to as the Initial System Narrative and provides a high-level description of the way the system operates. As with many simulation projects, modeling can start with assumed logic and placeholder values until they can be estimated or determined.

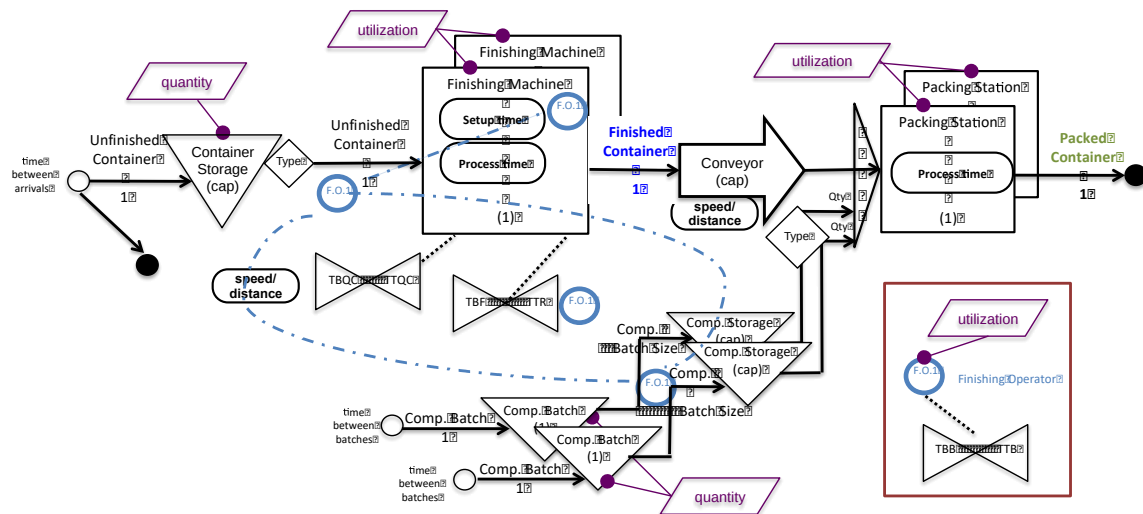
Initial System Narrative

Containers arrive to the finishing area at a yet to be determined rate. The number of types of containers and their properties are also not yet known. The arriving containers are loaded onto finishing machines by a finishing operator. If no machines are available, the containers wait in a buffer before being loaded. The size of the buffer and how the operator decides which container to process next needs to be studied. In order to maximize throughput, DPL is considering using the shortest-processing-time rule to load the finish machines, but is concerned some container types may wait too long for processing if it uses this approach.

The finishing process is automated; i.e., it does not need an operator. The finishing times, which will depend on the type of container, need to be determined. It is expected that a setup operation will be required on the finish machine if the current container type differs from the previous one produced on the machine. It is expected that the finish operator will do the setup. The number of finishing machines that are needed to meet forecasted demand and product mixes is also yet to be determined – the simulation will help determine this. Each finishing machine needs to run a quality check periodically and is subject to breakdowns that require repair. The quality check activity does not require any other resources; it primarily uploads data to a server. However, the repairs that result from a breakdown are expected to be performed by a finish operator.

After finishing, it is planned that containers move to a packing area via conveyor. At packing each container is loaded with a mix of components that depends on the container type. Components are delivered in batches and the finish operator will unpack the batches and make them ready for packing. After packing, packed containers move to the next process, storage waiting picking to fulfill customer orders.

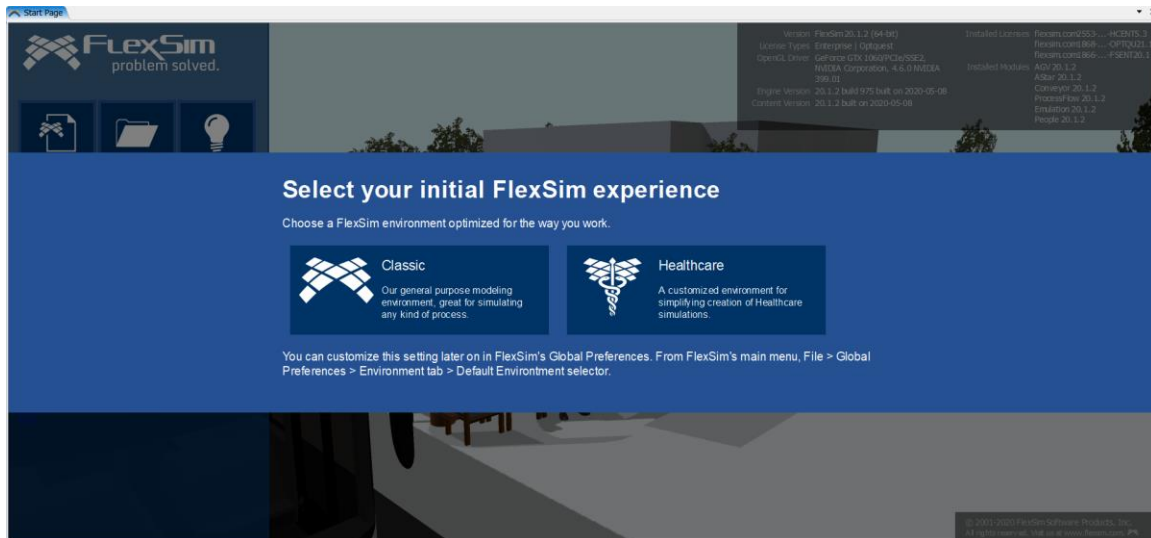
The narrative information is represented symbolically in the following OFD.



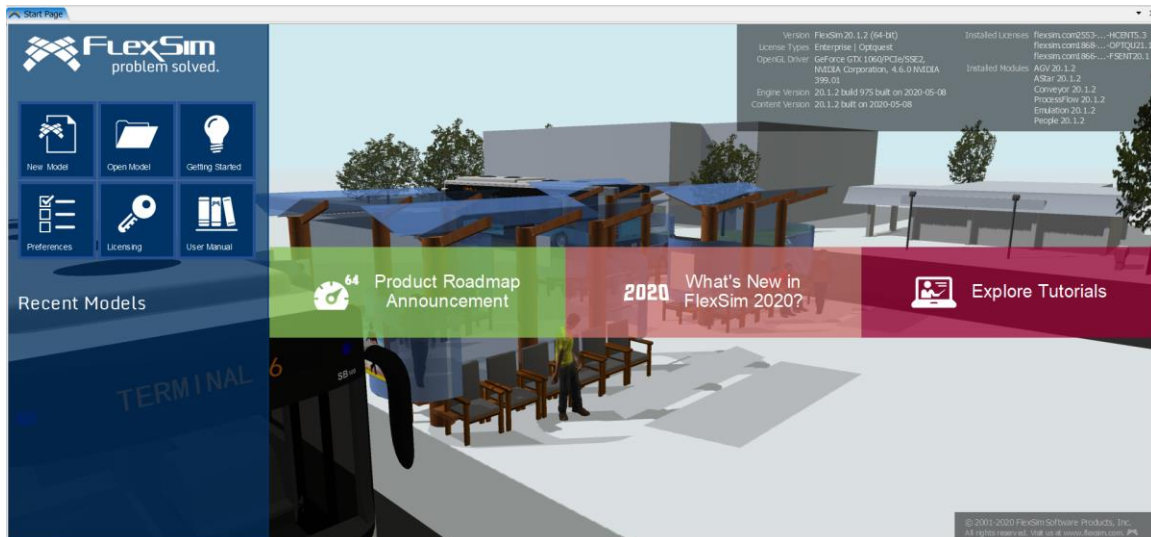
Note that the key performance measures of interest – denoted by the “slanted rectangle” symbol are the contents of the storage buffers for the containers and components (denoted as quantity), utilization of the finish machines, finish operator(s), and packing station. Also, the basic item flowing through the model changes, or is transformed by the operations, from a unfinished container to a finished container to a packed container. The finish operator moves material from the buffer storage to finish machines, performs the setup and repairs on the finish machine, and unloads batches of components in the packing area.

3 FLEXSIM'S MODELING ENVIRONMENT

When *FlexSim* is launched, by double clicking the *FlexSim* icon, the first choice is the desired modeling environment, as shown below, either **Classic** or **Healthcare**. This primer only addresses the Classic environment.



Classic FlexSim's Start Page is shown below. The main interface is in the upper left-hand portion of the page, where buttons are provided for starting a new model, opening an existing model, setting preferences, checking licensing information, and accessing the *User Manual*. Below the buttons are a list of recently-accessed model files. A model file can be launched by clicking on its name in the list.



The upper right-hand portion of the interface provides detailed information on the software version.

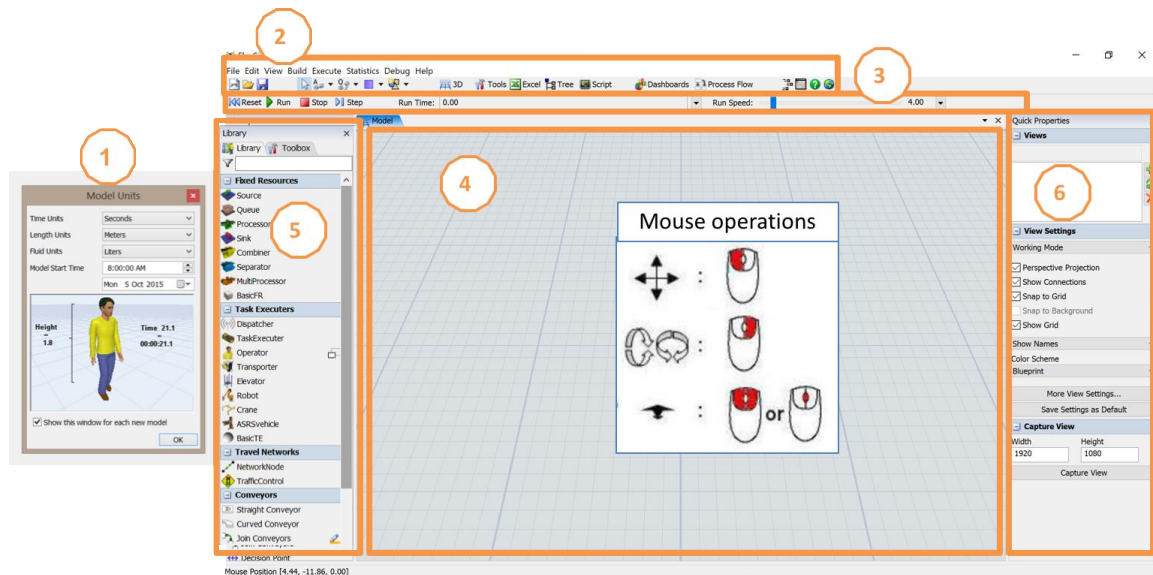
The tiled images in the main portion of the interface are links to FlexSim's website for more information on the software. If you are not connected to the internet, these links will not be displayed.

The 3D model image in the background, in this case the model represents a restaurant, changes from launch to launch, thus providing examples of application areas for where *FlexSim* can be used.

Once a model has been created, it can be accessed directly, without going through the Start Page, by double clicking on the model file.

The file extension for *FlexSim* models is fsm. *FlexSim* automatically creates a backup of the current model through an autosave feature. The default time between saves is 10 minutes, but this can be changed through Global Preferences. The autosaved file is named <your_filename>_autosave.fsm. Note that the model is only saved if it is not running; i.e., if a scheduled save occurs when a model is running, then the save does not occur.

The following figure shows the basic user interfaces in *FlexSim*. The numbers refer to the sections below where each interface is briefly described.



3.1 Time and distance units

When starting a new model, the first information that is needed is the model's units of measure. *FlexSim* is actually unit-less – i.e., simulations are conducted using general time units and distance units. Therefore, it is up to the modeler to specify the units appropriate for the system being modeled. A number of different model units are available through a drop-down menu for each unit of measure. Note that the default units are metric; i.e. length (distance) is in meters, time is in seconds, and fluid (volume) is in liters.

This first step involves a very important decision, since the specified **units cannot be easily changed in a *FlexSim* model once they are initially set**. However, there is Measure/Convert tool within *FlexSim* to help with conversions.

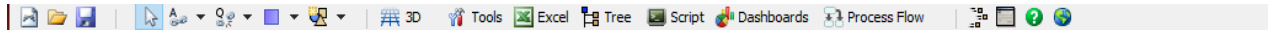
Model Start Time can be changed at any time, but in most cases, its value can be ignored when starting a model.

3.2 Main Menu and Main Toolbar

The Main Menu functions the same way as in most *Windows* applications. File, Edit, View, etc. – provide drop-down menu options for basic functions, such as opening and saving files or setting global preferences (File), undoing modeling actions or setting/viewing model parameters (Edit), viewing various user interfaces (View), executing a model (Execute), running experiments (Experimenter), and accessing the User Manual and license information (Help). The menu option Debug is for advanced users and is not discussed here.

Global Preferences is an important option under the File menu, especially for setting graphics card compatibility and customizing the tool bar.

The Main Toolbar is located below the Main Menu and is composed of a set of icons that provide shortcuts to basic *FlexSim* operations and capabilities, such as saving a model, opening additional 3D views, accessing the Toolbox, linking to *MS Excel*, creating or accessing Process Flow logic, etc.



3.3 Model execution toolbar

As the name indicates, this toolbar, located below the Main Toolbar, controls a model's execution. **Reset** suspends model execution, removes all flowitems from the model, resets all statistics, and sets the simulation clock back to 0. **Run** starts the execution of a model. **Stop** pauses a model run; the run can be continued by pressing the Run button again. **Step** advances a model one event at a time.



Run Time shows the current simulation clock time and its drop-down menu is used to set the planned stop time. There is no default planned stop time so unless suspended through Stop or Reset, a model will run indefinitely. The duration of a simulation run is in simulated time, not real time. Time in *FlexSim* is unit-less; it is given context through the user's specification of time units (e.g., seconds, minutes, days). If the user specifies the model units as seconds, then a Run Time of 10,000 is 10,000 seconds of simulated time (about 2.8 hours). Of course, it will not take 10,000 seconds to run because simulations can run much faster than real time. The execution time depends on the Run Speed.

Run Speed is like *FlexSim*'s "gas pedal" – it controls how fast a simulation runs. It can be set with the slider bar or via its drop-down menu. If a model's units are seconds, then a speed of 1.0 means the simulation is running in real time, i.e., one second of simulation time is one second of real time. Similarly, if the speed is set to 100, the simulation is running 100 times faster than real time. Therefore, if a model's Run Time is 100,000 seconds (about 28 hours) and Run Speed is set to 100, then the 100,000-second (28-hour) simulation will take 1,000 seconds (about 17 minutes) to run. The Maximum option in the Run Speed drop-down menu is the fastest a simulation will run based on the host computer's capability. *A simulation's speed can be adjusted as it runs to move quickly ahead in time in order to get to an interesting point in the model execution or slow down to carefully watch a certain behavior.*

The **Experimenter** provides another means to run simulation models and is discussed later in the primer.

3.4 3D model view window, modeling surface, and mouse operations

The model view window is *FlexSim*'s primary interface since this where models are constructed in 3D. The modeling surface is a gridded infinite plane in the x and y directions and located at 0 in the z direction. The x direction is to the left and right, the y direction is forward and backward, and the z direction is up and down.

The black cross on the grid denotes the origin, where $x=0$, $y=0$, and $z=0$.

The grid units reflect the length units that are defined when starting a new model.

As discussed in the next section, *FlexSim* objects are dragged into the model view and placed on the modeling surface. By default, object are placed *on* the modeling surface, but can be located above or below the surface based on the z-location parameter.

A mouse is used to navigate in the 3D model view. As shown in the figure above, the mouse movements are as follows: (1) the left mouse button moves the view left and right and forward and backward, (2) the right

mouse button rotates the view, and (3) using both mouse buttons, or the scroll wheel, zooms the view in and out.

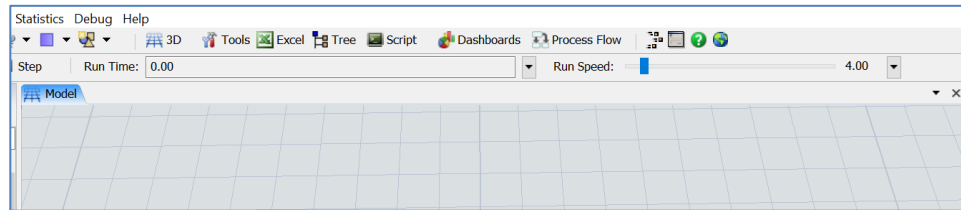


To reset the view of a model so that it is centered at the origin and is in a flat, two-dimensional plane:

(1) Right-click anywhere on the modeling surface, then (2) Select View >, then Reset View. This two-dimensional view is useful for placing objects in specific locations on the grid, connecting objects, and when the model appears to be lost somewhere in 3D space.



As shown below, the modeling surface is actually a view of the 3D model and is a tabbed interface in the modeling environment. As will be seen later, other things, such as dashboards and tables, can be a tabbed part of the modeling environment as well. The tabbed views can be removed by clicking on the X button in the top-right portion of the view. In the case of the Model view, it can be reopened by pressing the 3D button in the Main Toolbar. Therefore, *if you close the Model view, you only lose the view of the model, not the model itself!* Also, as discussed later, model views can be saved so that the saved view can be quickly recalled at any time.



A model view can also be created through the drop-down menu options on the View option on the Main Menu. Other views can be created from there as well, such as the Quick Properties, Drag-Drop Library, Toolbox, etc., which are described below.



Multiple model views can be created so that the model can be viewed from various perspectives. Again, views are created via the 3D button on the Main Toolbar. However, having too many views open can slow down model execution since the graphics on all of the views must be updated as a model runs. Another approach for having multiple model views is discussed in a later portion of the primer. Again, closing views do not affect the model. Even if all views are closed, the 3D model is still there, it is just not displayed.



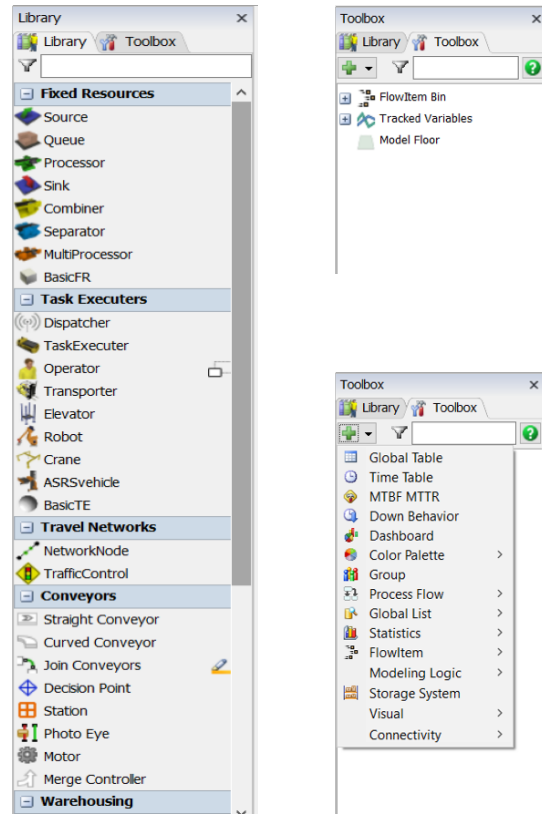
[section 4.2] **Simulation environment**

3.5 Object Library and Toolbox

As mentioned earlier, models are created in 3D by dragging *FlexSim* objects from the Object Library to the model view window and dropping them onto the modeling surface in the 3D view. The Object Library (also referred to as the Drag-Drop Library in the View drop-down on the Main Menu) groups objects by major categories, such as Fixed Resources, Task Executors, Conveyors, etc. A portion of the Object Library is shown in the figure to the right.

Other types of object libraries can be used in a model, such as special-purpose and user-developed libraries of custom objects. However, this is a more advanced feature and is only mentioned here to introduce a *FlexSim* capability.

The tab next to the Object Library is the tool library, referred to as the Toolbox, that provides access to a variety of modeling aides, such as data tables, time tables, dashboards, etc. An example is shown to the far right. The Toolbox shown at the top is the default list of tools that are used in a model, i.e., in a new model. The view of the Toolbox below the default is a list of all of the tools that are available. This list is accessed through the + icon. Many of these are discussed throughout the primer.



The various modeling objects and tools are introduced as the modeling examples are built. While *FlexSim* contains a wide range of objects and tools to facilitate modeling very diverse and complex systems, this primer only considers a subset of the capabilities, the key aspects that are needed to get started with *FlexSim*.

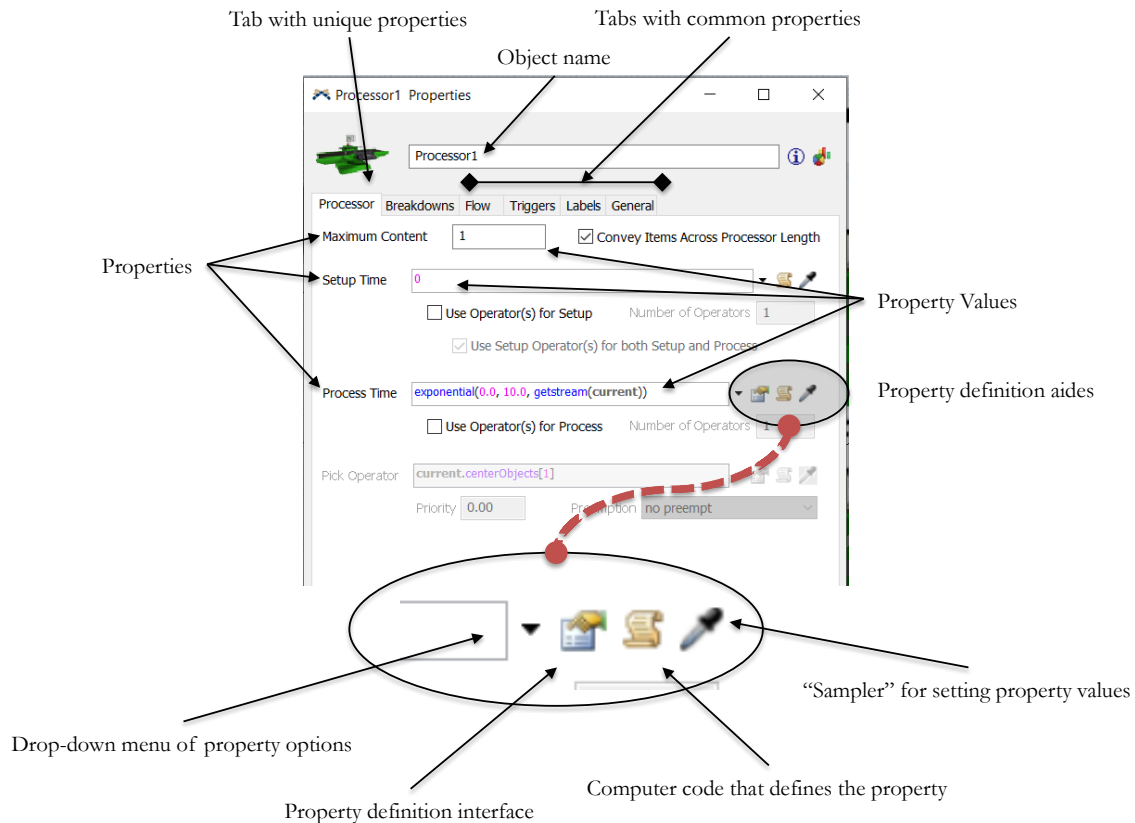
3.6 Object interface

As described earlier a significant part of model building, at least for basic models, is to drag and drop pre-built 3D objects to a modeling surface. The objects are selected, arranged and connected so that they represent the behavior of the system being simulated.

Each object represents a different functionality and contains numerous properties that are available to customize the object's behavior during a simulation. The properties are grouped by **tabs** on the object's user interface. Many of the tabs are the same across all objects since all objects can be customized by size and shape, means for routing items to and from the object, defining custom attributes or characteristics (called labels in *FlexSim*), defining special actions that occur within an object, etc. Having these common tabs greatly enhance learning and using the software.

Each object has one or more tabs that contain properties that are unique to its functionality.

The following figure, using the Processor object as an example, illustrates some of the common aspects of 3D object interfaces. For the Processor there is one unique property tab named Processor and the common tabs Flow, Triggers, Labels, and General. The Processor object has numerous **properties**, such as: Maximum Content, Setup Time, Process Time. The properties' corresponding *values* are in the text boxes; for the aforementioned properties, the values are 1, 0, and `exponential(0, 10, getstream(current))`, respectively. The values of the properties are what can be customized by the user/modeler.



To help provide values for the properties, *property definition aides* are found to the right of many property text boxes. One such set of aides is highlighted by the gray oval in the figure above. The aides are called out and identified at the bottom of the figure and briefly defined below.

- **Drop-down menu** of property options. Property values can be set by many means, such as: specifying a numeric constant, sampling from a probability distribution, looking up a value in a table, and following logic that is based on one or more states of the system. The drop-down menu is used to provide a list of means that are available for setting the property value.
- **Property definition user interface**. Once a means for setting a property value is selected, it likely requires specifying some parameters in order to be implemented, such as specifying the mean of a probability distribution or identifying which table to use for looking up a value. For each type of means, *FlexSim* provides a user interface that contains the required parameters that need to be specified.
- **Computer code** of the property. For more advanced applications, the predefined means for setting a property value may not be adequate. One way to further customize the model is to modify the underlying logic. Clicking on the scroll-shaped icon provides access to the underlying computer code, which may be modified using *FlexScript*, a subset of *C++*.

- **Sampler.** The eye-dropper-looking tool provides a convenient way to access properties of 3D objects by just clicking on the object with the Sampler and then selecting from the resulting set of applicable property values. This tool is illustrated later in the primer. It is especially useful when accessing information from 3D objects while modeling in Process Flow, *FlexSim's* custom logic builder.

3.7 Quick Properties panel

All of an object's properties are accessed by double clicking the object. However, a subset of commonly accessed properties is provided in the Quick Properties window. The window also includes some basic statistics that are updated as a model runs.

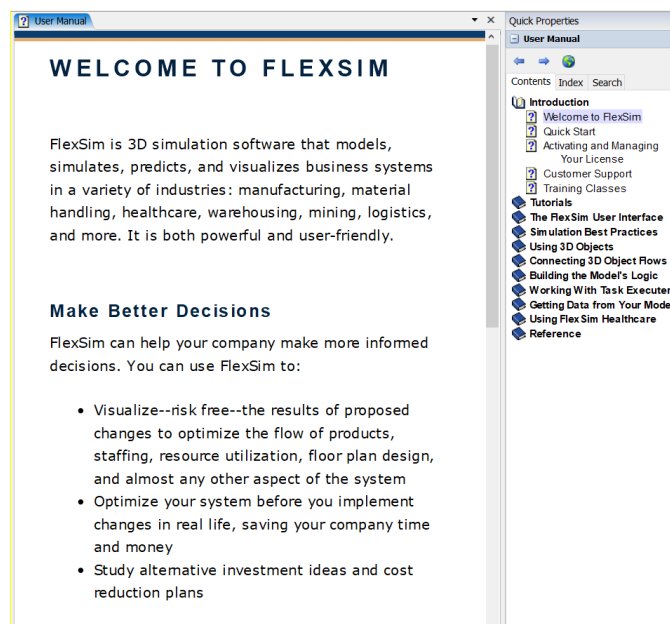
Clicking anywhere on the modeling surface brings up some of the viewing properties on the Quick Properties panel. For example, the Views section is used to create specific views of a model, such as an overall model view or a close-up of an individual object, and to easily switch between views. Also, in the View Settings section, the default Working Mode is used for model building, but Presentation Mode is useful for demonstrating a model since it hides connections, labels, etc.

3.8 Help

FlexSim offers many ways to obtain clarifications and more detail while using the software. Three of the primary means are introduced here.

3.8.1 User Manual

The first means to obtain additional information is via the Main Menu option **Help**. There are many resources available here, but the key one for new users is the **User Manual**. When this menu item is selected, the window shown below opens.



The left pane of the window provides the textual description of a topic.

The right pane, in the Quick Properties panel, contains three tabs. The first tab provides the table of contents for the full user manual. The second tab provides an extensive list, which can be searched, of indexed topics that are covered in the manual. The third tab provides the ability to search the User Manual using key words.

3.8.2 Context Help

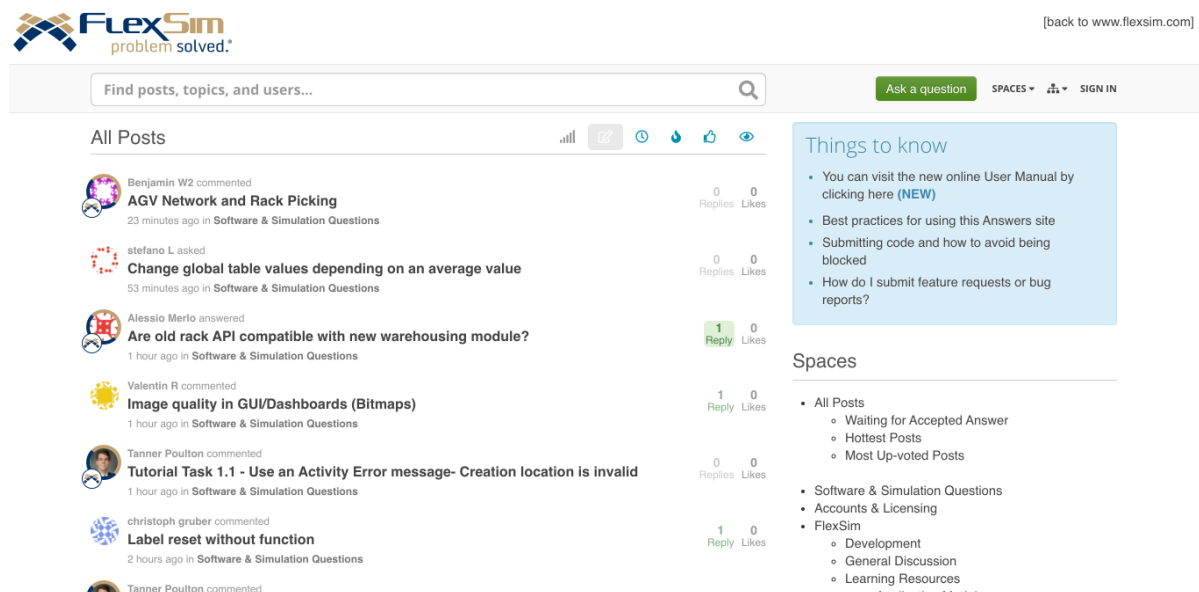
The second means to get help and further information is via the Help icon.



This icon can be found throughout the software. When the icon is clicked it goes to the relevant part of the User Manual depending on the context. One place where the icon is prevalent is in the lower-left corner of each tab of each object user interface. Clicking on it opens the User Manual with a description of that tab's functionality and an explanation of each property and parameter on the interface.

3.8.3 Answers

The third means for obtaining further information is the online help area that is referred to as **Answers**. It is a part of FlexSim's website and can be accessed at <https://answers.flexsim.com/index.html>. As shown in the snapshot of the site below, users can search for posts and topics and pose their own questions. This is a very active site and the posts are extensive, but the search capability makes it extremely useful.



Prior to using the site, one should review the best practice for using the forum.

<https://answers.flexsim.com/articles/22192/best-practices-for-using-this-answers-site.html>

All of these resources are very valuable for both new users and experienced users. Obviously, there are many means beyond this primer to learn about the vast capabilities of *FlexSim*.

4 GETTING STARTED WITH THE SIMPLEST MODEL

To start building a model of the system described in Section 2, start with the most simple model. This model is used to introduce the basic 3D modeling objects in *FlexSim* in terms of their functionality, structure, and properties, as well as obtaining basic output from the simulation. Once these fundamentals are explained, along with a step-by-step guide to customizing the objects and building the model, the primer describes how the basic model is expanded to better represent the system being considered and introduce more of *FlexSim*'s functionality and capability.

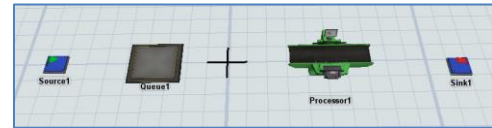


[section 4.3] Simulation components

A new model is started by double clicking the *Flexsim* program icon and selecting New Model from the Start Page.

For this case, use the default units, except set time to minutes; i.e., the model units should be meters, minutes, and liters.

Start with the most simple model by dragging the following objects from the Object Library to the 3D modeling view: Source, Queue, Processor, and Sink. The objects can be located anywhere, but arrange them similar to the figure to the right.



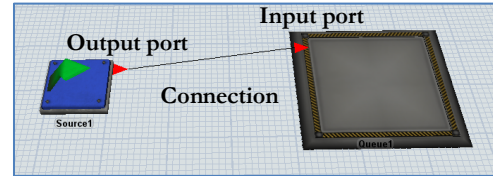
These objects represent the first part of the production process, the finishing operation, and provide the basic functionality for the simulation model. The Source generates flowitems; in this case it generates the containers that are to be finished. The Processor represents the finishing operation and the Queue is used to retain items that have arrived, but cannot access the finishing machine because it is busy. By default, a Processor can only process one item at a time. The Sink represents the next process and that is not modeled at this time. Items should move to the packing area, but that will be modeled later – we are starting small and simple.

While the four objects define the model's functional parts, the model is not yet complete - the relationship among the objects still must be defined. In this simple case, the objects are related by the sequential flow of the items between the objects. The item flow is defined by connecting the objects and, as with most aspects of modeling, there are several ways to do this. In *FlexSim*, there are several types of connections between objects; the item flow connection is referred to as an A-connection.

One approach for connecting objects for item flow, an A-connect, is to follow the sequence:

- Press the A key (notice the cursor changes from an arrow to links in a chain).
- Keeping the A-key pressed, click on the “from” object, where the flowitem comes from (the object becomes highlighted with a yellow box).
- Keeping the A-key pressed, drag toward the “to” object, where the flowitem is going (notice a yellow line emanating from the highlighted object).
- Click on the “to” object.
- Release the A key.

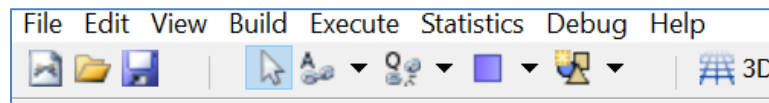
The result of an A-connect is that the two objects are connected with a thin black line as shown in the figure to the right. Also, two small triangles appear on the edge of the two objects. The one on the right-hand side of the Source is its **output port**, from where flowitems leave the object. The one on the left-hand side of the Queue is its **input port**, where flowitems enter the object. Every A-connect results in the automatic creation of two ports – an output port on the “from” object and an input port on the “to” object.



Be careful with the direction of the object connections since they affect the flow direction. The first object selected is the object *from* which items flow and the second object selected is where items flow *to*. Of course, this can be checked by looking at the ports connections as shown in the figure above.

If a connection is to be removed, follow the same steps as defined above for the A-connection, but hold down the Q key instead of the A key. Notice the cursor changes from an arrow to links in a chain with one link broken.

An alternative connection approach is to use the **Connect Objects (A)** icon on the main toolbar, next to the arrow icon and to the right of the new, open, and save icons, as shown below. With this method, the A key is not held down and objects are just clicked in the order that they need to be connected. The ESC key must be clicked once all connections have been made in order to return to Standard Mode, where the cursor appears as an arrow.



Similarly, the **Disconnect Objects (Q)** icon can be used to disconnect objects.

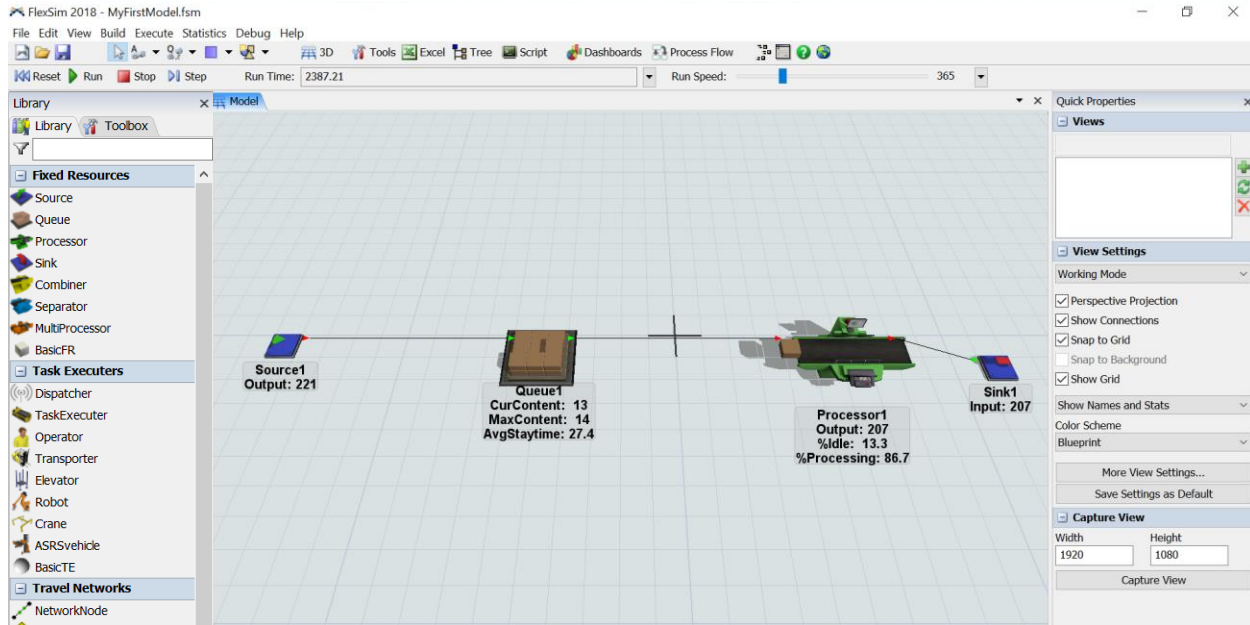


[section 4.7] Making connections



If a mistake is made in a model, the action can be undone, by using Undo command located under Edit in the Main menu.

Connect the objects, as shown in the figure below, then **Reset** and **Run** the model. It should look similar to the following figure. Items (boxes) should move across the object named Processor1 and should be accumulating in the object Queue1. If so, congratulations – you’ve built your first *FlexSim* model! Hopefully, this will be first of many helpful models for improving system performance.



Use the **Save** option in the **File** menu to save the current model. Use whatever file naming convention you prefer, e.g. MyFirstModel. *FlexSim* models have the filename extension *.fsm*.

By default *FlexSim* displays each object's name in the modeling view. This is the default **Show Names** setting in the Quick Properties panel (accessed by clicking anywhere on the modeling surface). This can be changed to either **Show Names and Stats** or **Show Nothing**. The **Show Names and Stats** option, as shown in the figure above, displays and updates a few basic statistics about each object as the model runs. As with most things in *FlexSim*, this can be customized.

A few things to note in the figure above, which is a snapshot of the model as it is running.

- The simulation time of the snapshot is 2387.21 minutes (**Run Time** in the Execution Toolbar) after the simulation started. This is nearly 40 hours of simulated time, yet the model only requires about a second or so of real time to run (at a Run Speed of 365, as shown in the figure).
- The Processor has been busy 86.7% of the simulated time, the current value of the **%Processing** statistic.
- Since the simulation began, 221 items have entered the model through the Source (its **Output** statistic) and 207 have left through the Sink (its **Input** statistic).
- The current number of items awaiting processing is 13 (**CurContent**). The maximum number waiting at any time since the simulation began is 14 (**MaxContent**). The average time an item spends waiting is 27.4 minutes (**Staytime**), which may or may not be acceptable system performance; it depends on the system. In a human-based system, it may not be acceptable to wait nearly a half hour for 10 minutes of service. However, in a manufacturing or logistics system, the wait time may be acceptable.

This model is running with all default parameters. While the basic flow logic is correct, the model is not using data that represents characteristics of the example system, such as inter-arrival times and process times. The next section begins the customization process.



Use the **Save Model As** option in the File menu to make a copy of the existing model so that it can be customized beginning in the next section. Again, use any file name, but in the primer, it is referred to as Primer_1-1.



Whenever a model is saved, *FlexSim* creates a second file with the same filename, but with the extension .fsm!. This is a backup file so that if the basic model file is corrupted or lost this file can be enabled by deleting the ! at the end of the extension.



FlexSim also automatically saves the model in a separate file every 10 minutes (the default time which can be changed on the Environment tab of the **Global Preferences** located in the **File** menu. The name of the backup file is the filename plus _autosave, e.g. Primer_1-1_autosave.fsm.



It is good practice to save a model frequently and to save the model as a new version, with a new name, whenever significant changes are to be made. This permits a roll back to a model that has been tested and works as desired if a mistake is made while making the change or if the change does not provide the desired result.

5 BASIC FIXED RESOURCE OBJECTS AND CUSTOMIZATION

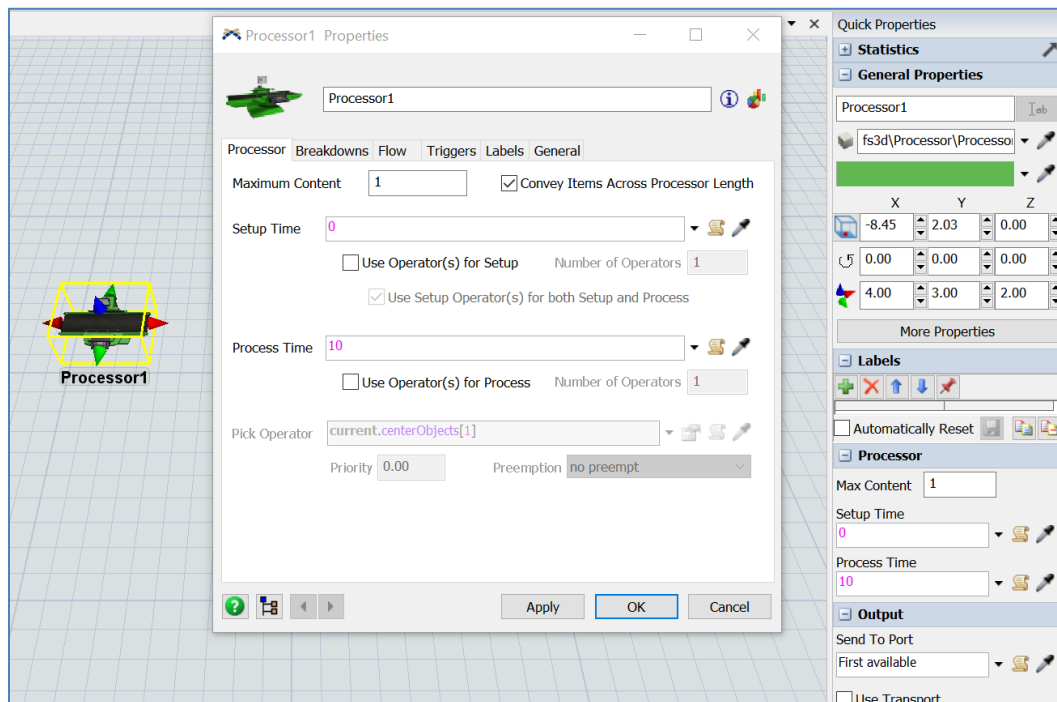
Each of the basic modeling objects that are used in the simple model are explored, and in doing so, the objects are modified so that the simple model becomes a better representation of the real system that is considered in the primer.



[sections 4.4, 4.7] Fixed resources, Making connections

5.1 Basic object properties and structure

The objects are customized by modifying their properties, which are accessed by double clicking on the object in the model view or through the Quick Properties panel when an object is selected in the model view. The figure below shows both interfaces – the Quick Properties are available in the panel to the right and the main or detailed properties interface is shown in the center of the figure.



The Quick Properties interface provides access to a commonly-used subset of an object's properties. It is accessed by selecting an object, i.e., just by clicking or selecting it and without opening.

However, in this primer, most of the properties will be changed through the main, detailed interface that is accessed by double clicking the object. All of the properties of an object are organized on tabs. For the Processor, as shown above, all of its properties are contained on six tabs.

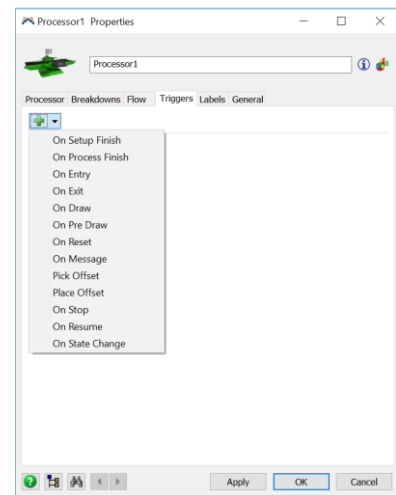
The number of object properties in *FlexSim* is extensive, thus providing a great amount of modeling flexibility and capability. Since the purpose of this primer is to introduce the basics of how *FlexSim* works and some of its capabilities, only the most basic and salient features and properties are presented. Therefore, it is well beyond

the scope of this primer to discuss every property on an object interface and every object and feature in *FlexSim*. With the foundation provided in this primer, the User Manual can be used to understand other properties and parameters.

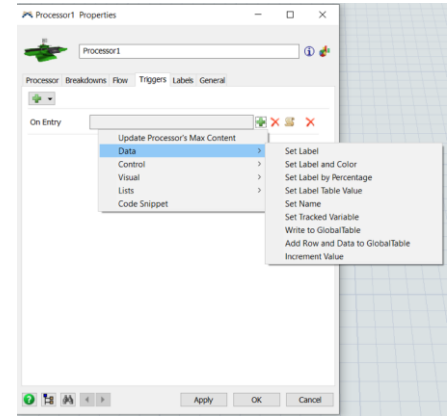
It is important to note the buttons at the bottom of the object interface. **Apply** saves any changes, but does not close the window. **OK** saves any changes and closes the window. **Cancel** does not save any changes and closes the window. Avoid closing the object windows with the X in the upper-right corner of the window – it functions the same as **Cancel**, thus not saving any changes made in the properties in the window.

One thing to note as the various objects are explored is that the basic structure of the different types of objects is the same. Object properties are grouped using tabs. All objects have the tabs defined below and one or more tabs that contain properties unique to that type of object. Having a common user interface for object properties greatly enhances learning *FlexSim* and its overall ease of use.

- General tab – basic object information, including:
 - Appearance for specifying 3D shape, color, position, size, etc.
 - Flags for controlling what is displayed and the object's accessibility.
 - Ports for displaying and controlling what objects are connected to the object's ports.
- Labels tab for defining custom attributes that are used or updated during a simulation; there are multiple types of labels, numeric, string, pointers, arrays, etc.
- Triggers tab for invoking optional actions that can occur as a model runs.
 - Triggers have many and diverse uses and are grouped both by *when* the action occurs and by *what* type of action occurs. The type of trigger varies by the object, but many of the triggers are common across all of the objects.
 - When an action specified by a trigger is executed, it is said that the trigger “fires.”
 - Triggers are first specified by *when* they occur, such as **On Reset**, **On Stop**, when an item enters or exits the object or both (**On Entry** and **On Exit**), etc. The figure to the right shows when triggers can fire in a Processor. For example, when a setup operation is completed (**On Setup Finish**), when the basic process is completed (**On Process Finish**), when a flowitem enters the Processor (**On Entry**), etc.



- Once *when* a trigger fires is defined, then *what* action is to occur is specified. The figure to the right shows *what* actions are possible when an **On Entry** trigger fires. The actions are grouped by the type of actions: **Data**, **Control** (e.g., open or close object ports), **Visual** (e.g., set color or size), **List** (e.g., push to or pull from lists), and **Code Snippet**. In the case of the figure to the right, for an **On Entry** trigger on a Processor, a variety of **Data** actions are possible, such as **Set Label**, **Set Label Table**, **Set Name**, etc.



- All objects have **OnReset**, **OnMessage**, and **Custom Draw** triggers.
- Note, all Fixed Resource objects have both **OnEntry** and **OnExit** triggers, except the Source only has an **OnExit** trigger (nothing enters a Source); similarly, the Sink only has an **OnEntry** trigger (nothing exits a Sink).
- All Task Executors have **OnLoad** and **OnUnload** triggers; these are analogous to the **OnEntry** and **OnExit** triggers for Fixed Resources.
- All objects have triggers related to their type, e.g., **OnCreation** in a Source object, **OnProcessFinish** in a Processor, or **OnResourceAvailable** in an Operator object.

All Fixed Resources have a Flow tab that has three major functions:

- Control *to which output port a flowitem should be sent* when it leaves an object, e.g. based on the downstream object(s) availabilities or queue size, an item's attribute or label value, or a table value.
- Specify *whether an item uses a transporter* to reach the next object. If a transporter is used, it is requested based on specified criteria. The item waits in the current object until a transporter arrives and loads the item. Therefore, if an object's capacity is one item, then a new item is not moved into the object until the current object exits via a transporter (if required).

Note that if no transport is used, then an item moves between two Fixed Resource objects in zero simulation time regardless of the distance between the two objects.

- Enable *pulling items into the object*. Normally items move through a model by being "pushed" from one object to another; i.e., the current object determines to which subsequent object an item moves. Pulling enables an object to determine which item it processes next based on specified criteria.

Each object has a name. Since *FlexSim* must ensure each object has a unique name, it automatically assigns a name when an object is created by selecting it from the library and dropping it into the 3D modeling view. When a model has multiple instances of a type of object, even with a relatively small number of objects, it becomes difficult to recall which object does what in a model, e.g. the difference between Source1 and Source2. Therefore, each object should have a meaningful name that reflects its role in the model; e.g. ContainersArrive is a much more meaningful name than Source1. The naming is for the modeler's benefit, and others who access the model, not for the software.



Begin with the basic model from the previous section, named Primer_1-1, and **Save As** Primer_1-2. Basically make a copy of the model so that it can be customized and the original model is retained.

5.2 Source object

The Source is a modeling boundary, a starting point for what is being considered in the system. For this example, what happens to the containers prior to arriving to the finish area is not a concern at this time.

Objects should be named in a meaningful way. Therefore, change the object's name from Source1.

- Name the Source object **ContainersArrive**.

Information about the flowitem being created is specified on the Source tab. Flowitems are created in one of three ways, based on: (1) inter-arrival times, (2) schedule, or (3) sequence. The default value for the **Arrival Style** property is **Inter-Arrival Time** and the default is used in this example. The **Arrival Schedule** option for the **Arrival Style** property is discussed later in the primer.

The time between arrivals is defined by the **Inter-Arrivaltime** property. The default is:

exponential(0, 10, getstream(current))

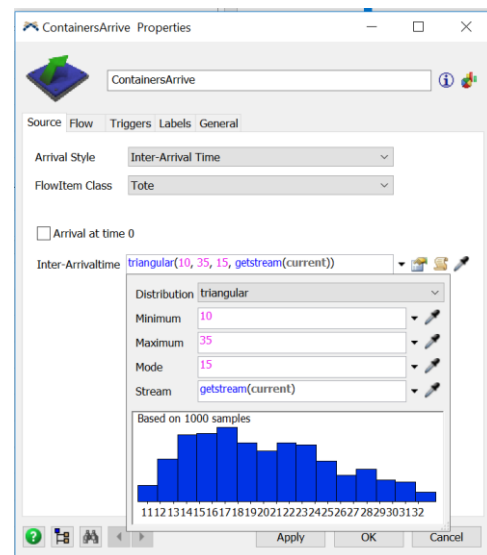
i.e., each inter-arrival time is obtained by a random sample from an exponential probability distribution with a mean of 10 time units, 10 minutes in this example.

The property's value can be set from a variety of drop-down menu options. For this example, and as shown in the figure to the right, change the inter-arrival time distribution.

- For the **Inter-Arrival Time** property, select the **Statistical Distribution** option and then the **Triangular** option in the submenu.

Note that there are many distributions to choose from in *FlexSim*. Each distribution has a different set of parameters. The triangular requires three parameters - the minimum, maximum, and most-likely (mode) times between arrivals.

- As shown in the figure to the right, enter the triangular distribution's parameters for this example. The minimum, maximum, and most-likely (mode) values are 10, 35, 15, respectively.



In order to implement a distribution, the user interface provides the specified values to a *FlexSim* command; so that it can be executed; in this case, it translates the interface values to:

triangular(10, 35, 15, getstream(current))

These specified parameters result in an average time between arrivals of 20 minutes, the sum of the three parameters divided by three. In general, the mean of a triangular distribution is the simple average of its three parameters.

An average time between arrivals of 20 minutes corresponds to an average arrival rate of 3 customers/containers per hour. Rates are the reciprocal of time values; in this case, the average arrival rate is

60 minutes/hour $\div$ 20 minutes/customer = 3 customers/hour. Simulation software typically use average times rather than average rates.

The last parameter in specifying any probability distribution in *FlexSim* is the number of the random number stream to be used for this source of variability. This is implemented through the command, `getstream(current)`. Discussion of random number streams is beyond the scope of the primer. Also, commands are discussed later in the primer. However, it is fine to use the default for the **Stream** property. It is good general practice that stream values be unique for each source of variability in the model – this is handled automatically by *FlexSim* through the `getstream(current)` command.



Discussion of selecting probability distributions for interarrival times, process times, failure times, etc. is beyond the scope of this primer. *ExpertFit*, which comes with a *FlexSim* license, is a good tool for fitting data to distributions.



Also, it is generally a good idea to use bounded distributions so that the model does not use unrealistic, extreme values that can result from such distributions as the normal, exponential, etc.



[Chapter 8] Modeling Randomness

The type of flowitem that is created by the Source is specified by the **FlowItem Class** property. The default type is **Box**, but there are other options in the drop-down menu. Also, it is easy to create custom flowitems. This is done later in the primer.

➤ For this example, select **Tote** as the **Flowitem Class** value.

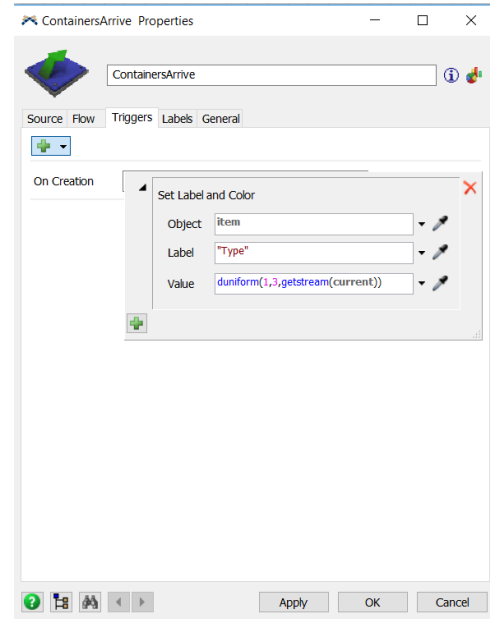
In this example, there are multiple types of containers, all of the same class; i.e., they all use the same flowitem container, **Tote**. For now, assume three equally-likely types; this will be changed later in the primer. To easily distinguish the types in the model they will each have a different color. To model this in *FlexSim*:

- In the Source object, use the **OnCreation** trigger and select **Data** and then select the **Set Label and Color** option. Use the default properties as shown in the figure to the right.

As mentioned earlier, attributes in *FlexSim* are called *labels*. The **OnCreation** trigger that is shown to the right creates a label named **Type** for each item that is generated at the Source. In this case, the value of **Type** is the result of a random sample from a discrete-uniform distribution with possible values of 1, 2, or 3, with each value equally likely. This action is accomplished by the *FlexSim* command

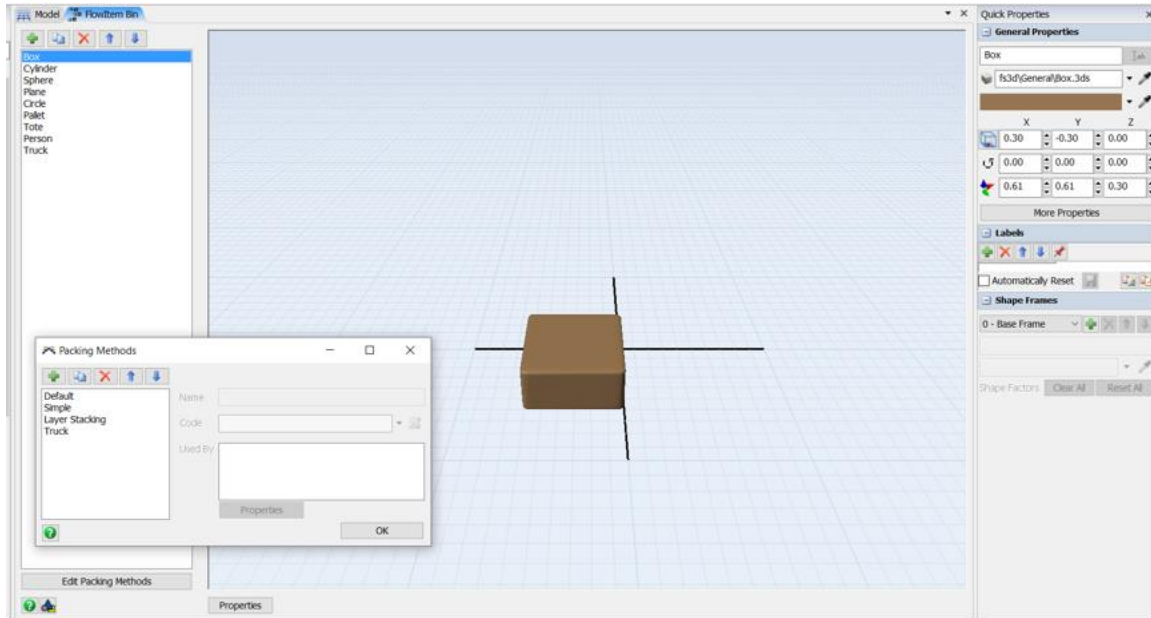
duniform(1, 3, getstream(current))

This trigger also sets the item's color based on its **Type** value. The default color scheme in *FlexSim* is 1=red, 2=green, 3=blue, 4=yellow, etc. Of course, a custom color scheme can be defined.



5.3 Flowitem Bin

Flowitems are customized through the **FlowItem Bin**, which is accessed either via the ToolBox or its icon located on the right end of the Main Toolbar. An example of the FlowItem Bin is shown in the figure below. The default classes of flowitems are shown in the left-hand portion of the interface. The lower-left corner shows the packing methods that are available for the container flowitems., as defined below. The right-hand panel in the figure below shows the Quick Properties of the selected flowitem.



There are two general categories of flowitems – Basic and Container.

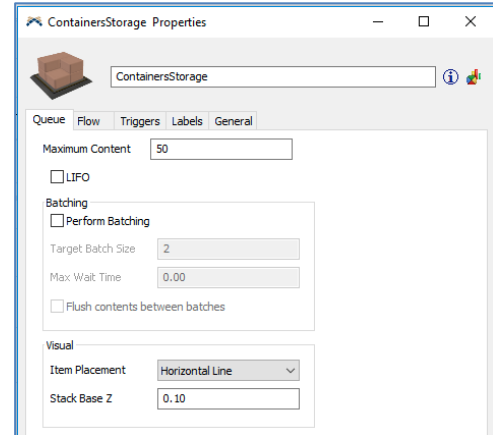
- Basic flowitems are simple shapes, such as **Box**, **Cylinder**, **Sphere**, **Plane**, **Circle**, and **Person**. Their visual appearances – such as size, shape, rotation, color - can be customized.
- Container flowitems – **Pallet**, **Tote**, and **Truck** – are special in that they can contain basic flowitems and can be packed in various ways.
 - The **Default** packing method is used by the **Pallet**. Items are stacked on top of the container filling up as much available space as possible before moving to the next level. This is similar to the way items are stacked in a Queue by default.
 - The **Simple** packing method is used by the **Tote**. This is similar to the **Default** method except flowitems are packed *into* and not *onto* the container.
 - The **Layer Stacking** method considers the uniformity of flowitem sizes. When flowitems have a consistent height (z direction), they are put on one layer of the container. As soon as the height of a flow item differs from the others, it is placed on a new layer. The width (x direction) or depth (y direction) can be checked instead of z.
 - The **Truck** packing method is used by the **Truck** class of flowitem. Within the Truck item, other flowitems are first stacked on top of each other in the forward end of the truck and the stacking moves toward the back.
 - Each of the packing methods can be customized through *FlexScript*.

5.4 Queue object

The Queue object delays an item if no downstream object is available to process an item. In this case, if a container arrives to the system and the Processor is busy, it waits in the Queue.

For this example, the Queue properties are changed as follows and as shown in the figure to the right.

- Name the Queue object **ContainerStorage**
- Change the **Maximum Content** property from the default value of 1000 to 50. By default, the queue size is assumed large (1000), but in this example, it is assumed that the maximum realistic quantity is 50. This will likely change as the system is designed and the space available for storage becomes clearer.
- Change the **Item Placement** property from the default value of **Stack inside Queue** to **Horizontal Line**. By default, items are stacked in the queue by filling up as much available space as possible before moving to the next level. *The quantity in a layer depends on the physical size of the Queue and of the flowitems.*



5.5 Processor object

The function of a Processor is to invoke a planned delay on items flowing through a model.

- Name the object **FinishMach_1**; it is anticipated that more than one finish machine will be needed.

Each finish machine processes one container at a time, which is the default value for the **Maximum Content** property. However, note that a Processor can process multiple flowitems concurrently by changing this property's value.



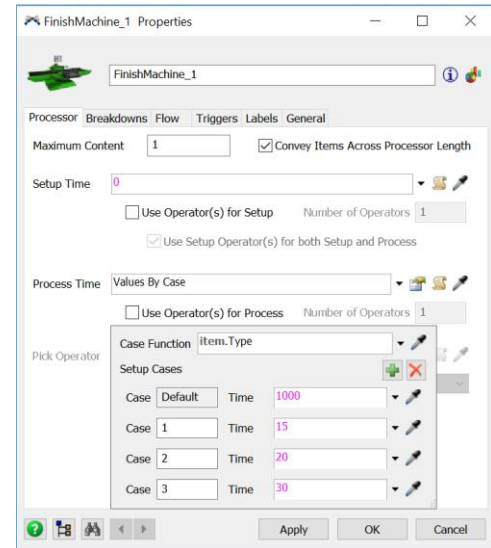
Using the **Maximum Content** property is a quick and convenient way to represent multiple resources by using only a single object. This is handy when building a simulation model to obtain a rough estimate of system requirements. However, modeling multiple resources through a single object precludes modeling certain system aspects, such as downtime on individual resources, operator travel to specific resources, processing specific types of items on specific resources, etc.

The planned delay is specified by the **Process Time** property. The default is a constant 10 time units. In this example, the finish time depends on the type of container; i.e. the finish time for Type 1 is 15.0, Type 2 is 20.0, and Type 3 is 30.0. This is implemented through the **Values By Case** option. In this example, each case corresponds to a type; therefore, three cases are created.

To model the situation where process time depends on product type, perform the following steps:

- Select the **Process Time** property, then select the **Values By Case** option from the drop-down menu.
- Create 3 cases, in addition to the default, by using the + button; then set the values as shown in the figure to the right.

The logic works as follows. When an item enters this Processor and needs a process time, *FlexSim* checks the value of the label Type and it becomes the case number. Therefore, if an item's Type value is 2, the time specified for Case 2, 20.0, is used for the process time. The Default Case value is used if an item has a Type value other than 1, 2, or 3.



The notation in the **Case Function** property is the way *FlexSim* refers to a label. In general, it is referred to as **dot notation** because a dot or “.” separates parts of the notation, i.e. object.LabelName. In the case of item.Type, the object is the flowitem, referred to as item, and the name of the label that is being considered is Type.



The default process time in the **Values By Case** logic is set to 1000, or any large number. This helps the modeler to remember to add a Case if a new type of container is added to the model. This way, if a fourth type is added in this example and this property is not updated with a new Case, then the item will be delayed a long time at the Processor and things will back up in the model. This should prompt an investigation as to the problem and hopefully the failure to update the process time is discovered and remedied. If a small value is used for the default, this oversight may not be caught and the new container type will use an erroneously low value, thus affecting the results.

Since the three types of containers are equally likely, the average processing time is 21.67 minutes (average of 15.0, 20.0, and 30.0). Recall, the average time between arrivals from the source is 20 minutes. This means the single finish machine will be overloaded since the planned utilization is greater than 100%, on the average ($21.67/20.0 = 1.083$ or 108.3%). The situation will actually be worse since there is variability in the system due to the type of container, time between arrivals, and the finish machine not being available 100% of the time due to downtime. One way to remedy the large queue contents is to use another finish machine, which is done in Part 2 of the primer.

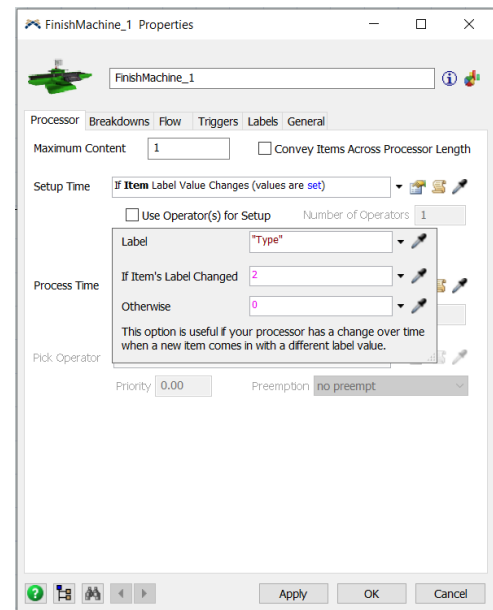
In addition to the process time, there is a *setup* operation. The setup operation is separated from the process time for two reasons:

- (1) An operator will perform the setup operation, but the machine will run on its own once it is set up. (This will be incorporated into the model in the next part of the primer.)
- (2) The setup operation only needs to be performed when the type of item (container) changes.

The time to perform the setup operation, when it needs to be performed, is assumed to be a fixed two minutes.

The setup logic is implemented into the *FlexSim* model as follows and as shown in the figure to the right:

- Select the **Setup Time** property on the Processor object's Process tab, then select the drop-down menu option **If Item Label Value Changes**.
- Update the resulting menu's property values as shown in the figure to the right and as described below.
 - **Label:** no change, use the default **Type**.
 - **If Item's Label Changed:** Set this property's value to 2.
 - For the property **Otherwise**, change the default value 5 to 0.



5.6 Sink object

The Sink object removes flowitems from a model; therefore, it is the opposite of a Source. However, this functionality is much less complex than that of a Source. There is not as much involved in removing items from a model, compared to creating them, which involves defining what to create, with what properties, when to create them, etc. The Sink is another modeling boundary; the stopping point for what is being considered in the system.

- The only change that is made to the Sink is its name, **NextProcess**.



If you haven't already done so, save the model. Recall, it is good practice to save often.

6 BASIC MODEL OUTPUT

Simulation models are developed to assess the dynamic behavior of an operations system. Therefore, there are multiple means to access the results or output from a simulation model, a few simple ones are described here. Output is usually obtained from *FlexSim's* Experimenter. It, among other things, is used to:

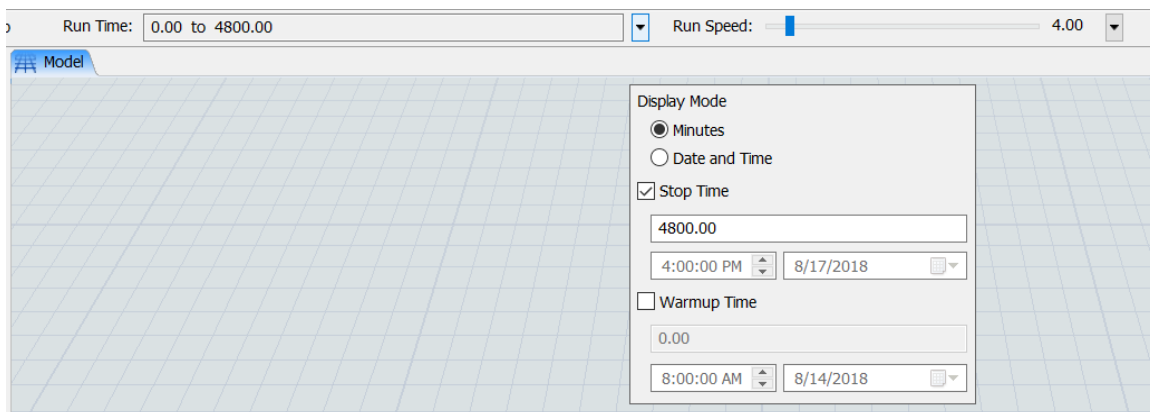
- Execute multiple experiments in a single run.
- Control experimental conditions, such as number of replications and warm-up period (advanced topics discussed later in the primer).
- Obtain statistical estimates of system performance measures.
- Directly link to OptTek's *OptQuest* optimization software.

However, for now, only a few very basic output measures are discussed here.



[section 4-10] **Single-run statistics**

Consider an 80-hour run of the simulation model (equivalent to ten 8-hour shifts). In model terms this is 4,800 minutes; recall, the model's time units were defined to be minutes when the model was initially created. The figure below shows setting the model's run time and is followed by the instructions.



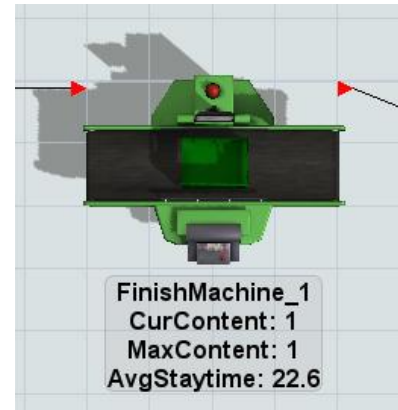
- Select the dropdown menu on the **Run Time** property that is located on the model's Execution Toolbar.
- Check the **Stop Time** box and enter 4800 is entered into the box below it.

So that model finishes more quickly, the **Run Speed** can be adjusted higher than the default value of 4 by using the slider interface.

6.1 Object statistics and Quick Properties

A few basic output measures are provided for each object on the modeling surface, as shown in the figure to the right. These values are continually updated as a model runs. At the moment when this screenshot was captured (at the end of a 4800-minute model run) the Processor, named FinishMachine_1, was processing an item (CurContent), the most it processed at one time was 1 (MaxContent), and the average setup and process time is 22.6 minutes (AvgStayTime).

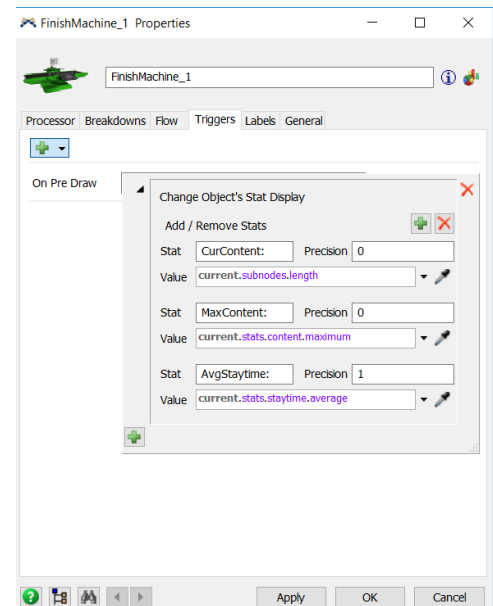
The default object statistics can be displayed for all objects through the **Show Names and Stats** setting in the Quick Properties panel. This is explained below.



As with just about everything in *FlexSim*, these measures can be changed. As shown in the figure to the right, this can be done through a trigger. In order to access the parameters:

- Select the **On Pre Draw** trigger on the object's Triggers tab
- Then, select the **Change Object's Stat Display** option on the object's **On Pre-Draw** trigger.

Discussion of how to add or modify the statistics is currently beyond the scope of the primer.

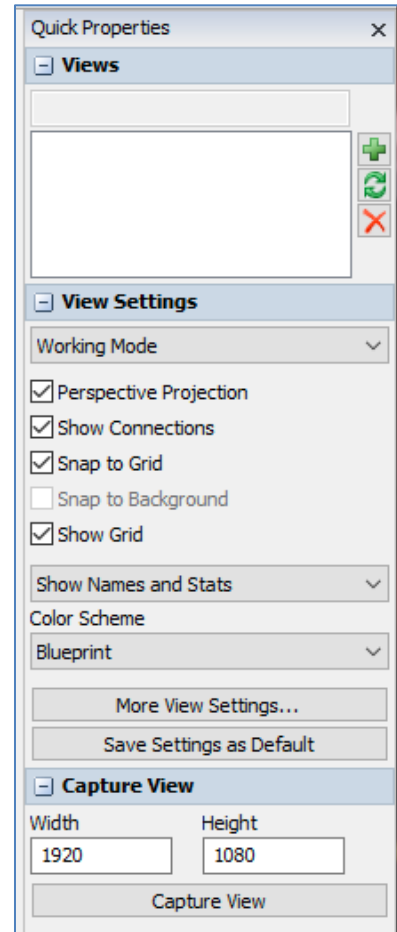


What is displayed on the modeling surface as a simulation runs is controlled by the **View Settings** section in the Quick Properties panel. An example of the panel is shown to the right.

This panel is accessed by clicking anywhere on the modeling surface in the 3D modeling view. Three display options are available: **Show Names and Stats**, **Show Names**, and **Show Nothing**.

The panel is also used to:

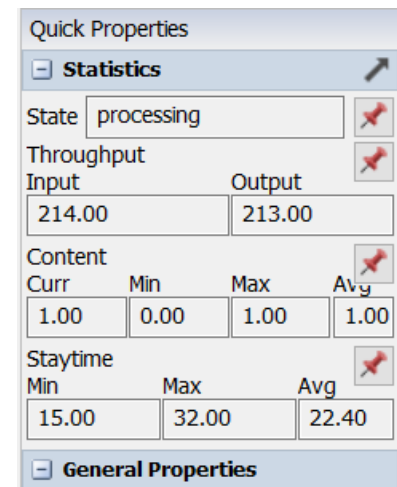
- Create views in order to easily switch the model's display to predefined views, such as an overall view of the model, a close view of a particular object, etc.
- Change the basic viewing mode from the default, **Working Mode**, to **Presentation Mode**, which removes the grid and connection lines and uses perspective projection.
- Control the view settings individually by checking or unchecking the check boxes.
- Access additional, more advanced, view settings through **More View Settings ...**
- Take a snapshot of the model, **Capture View**, and save it as a graphics file with a png extension.



If an object is selected, a few basic output measures are displayed on the **Statistics** section of the Quick Properties panel, as shown in the figure to the right. These values are continually updated as a model runs. The panel provides information on the object's:

- **Current State**; in this case, when the simulation ended, the object was busy processing.
- **Throughput**, how many items entered and left the object. In this case, 214 entered, 213 left; thus, one is currently being processed.
- **Content**, how many items are currently in the object and the minimum and maximum number to date that have been in the object, as well as the average number.
- **Staytime**, how long items have stayed in an object, in terms of the minimum, maximum, and average times. Since the process times are constant, the **Min** value, 15.0, is the time to process container Type 1; and, the **Max** value, 32.0, is the time to process container Type 3 plus a 2-minute setup time. The average value for all items is 22.40.

The pushpin icon is used to “pin” that statistic for that object onto a Dashboard, which is described in the next section.



6.2 Dashboard

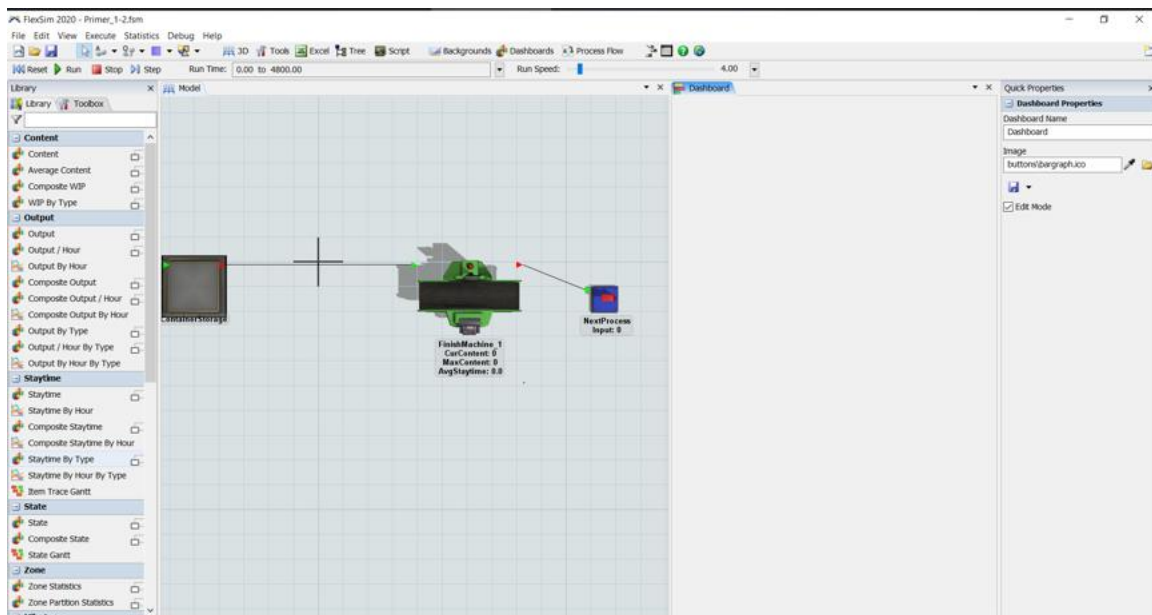
Compared to considering a single value at a snapshot in time, like the on-object and Quick Properties values described in the previous section, charts and graphs are very helpful for assessing the dynamics of a system. *FlexSim* provides a wide variety of charts and graphs through the Dashboard tool. The Dashboard is either accessed from the Main Toolbar or from the Toolbox (Tool Library). A model may have multiple Dashboards that capture various aspects of a system's behavior.

Three example charts from the Dashboard are described here, two types of time-series plot (a value plotted over time), and a histogram. Prior to describing how to create a particular chart, some background is provided on the Dashboard itself.

Charts and graphs are created on a Dashboard.

- Click the **Dashboards** icon on the Main Menu and select **Add a Dashboard**. If you create multiple dashboards, then they can be selected either from this Main Menu option or can be selected from the Toolbox.

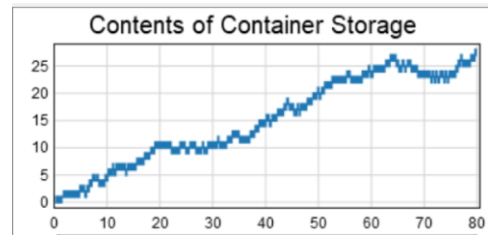
As shown in the figure below, a “blank” dashboard is automatically created in the right portion of the screen and a library of chart tools and templates is created on the left side. The blank space to the right is the workspace to create a set of charts and graphs.



The library of chart options on the left in the figure above offers many templates for commonly-used charts. Of course, as is always the case in *FlexSim*, custom charts can be created. However, for now, the primer uses the predefined ones. Notice that if you click on the grid in the 3D space, then the library changes back to the 3D objects. The library changes based on what part of the *FlexSim* environment you are in – 3D, Dashboard, Process Flow, etc.

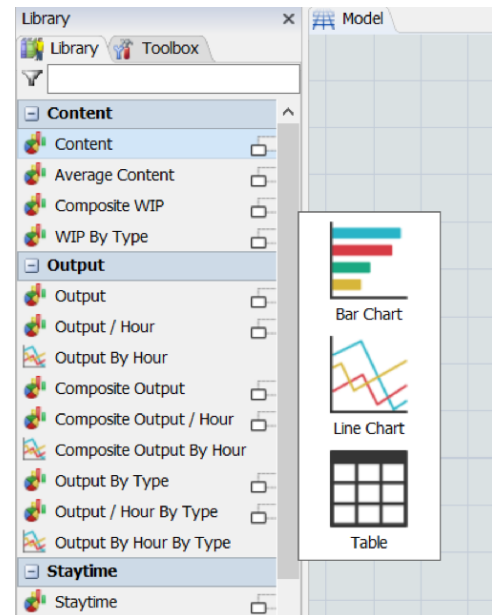
Time-series plot

The first type of chart that is considered in this example, as shown to the right, is a system-level plot of the total content of the ContainerStorage queue over the duration of the simulation (4,800 minutes). The chart clearly depicts the dynamics of the system. It also indicates that, as expected, the Queue is continually growing over the duration of the simulation. Recall, this is because the average process time is longer than the average time between arrivals. Conversely, the average service rate is less than the average arrival rate.



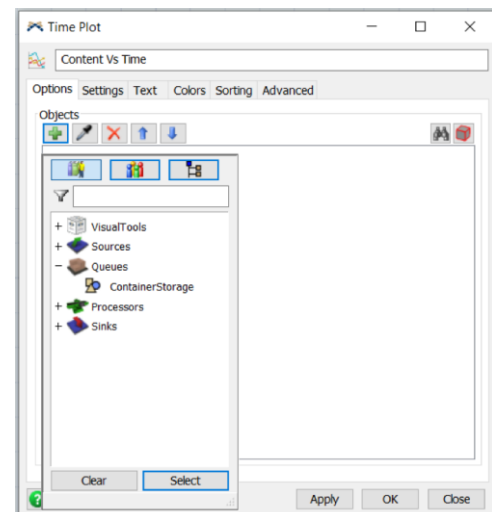
To create the time-series chart:

- As shown in the figure to the right, select the **Content** chart type in the **Content** section of the Dashboard Library; then, select **Line Chart** from the pop-up menu.
- Drag the Dashboard Library item onto the Dashboard panel where it can be sized and positioned.
- As shown in the figure to the right, use the Time Plot interface to rename the chart from Content Vs Time to something more meaningful for the example, **Contents of Container Storage**.



The only required input in the selection is to identify which object(s) to plot. Plots for multiple objects can be overlaid on the same chart. In this simple example, only one object is considered.

- As shown in the figure to the right, on the Options tab, in the **Objects** section, click the + button and choose **Select Objects**. This generates a list of all of the objects that are currently in the model by category (Sources, Queues, Processors, etc.).
- From the **Queues** category, select the object ContainerStorage. Currently the only queue in the model. Notice that the object is highlighted. Now press the **Select** button. The ContainerStorage object is now displayed in the **Objects** section.



- The only other parameter that should be changed for now is the **Time Axis Mode** property on the Settings tab. Change the value from the default **Show exact time and date** to **Show Duration**. Also, change the displayed units from **Seconds** to **Hours**.
- Since only one value is being plotted and the chart title is descriptive, uncheck the **Show Legend** box (also on the Settings tab).
- The default settings for all of the other chart parameters are fine for now. Therefore, press OK to close the interface.
- Relocate the chart on the Dashboard.
 - Notice when a chart is selected, it contains a double-lined frame around it with “handles.” Handles are small squares at the corners and midpoints of the sides of the charts. Also, notice that when the cursor is over the perimeter frame, it changes from the standard arrowed pointer to two crossed double arrows.
 - Left click with the mouse anywhere on the perimeter frame of the chart to move it around and place it anywhere on the Dashboard window.
 - Similarly, left click one of the handles and drag it to resize the chart. Notice the cursor changes to arrows depicting what direction is being resize – horizontal, vertical, or both (diagonal).



The “Content” chart that is introduced above indicates the importance of considering how long to run a simulation. Had the model only been run 480 minutes (8 hours), the maximum contents would have only been about five items, compared to nearly 30 when run 4800 minutes (80 hours). Different decisions could have resulted based on the different run lengths. Discussion of this topic (how long to run a simulation), while very important, is beyond the scope of this primer.



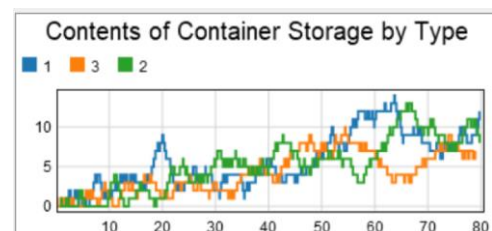
The “Content” chart above also illustrates the dynamics of the simulation. Due to variability and interactions the system could behave quite differently in another 4,800-minute time period, just as in the real system, behavior varies day to day or week to week. This illustrates the importance of running a simulation model multiple times, referred to as **replications**, and combining the results, e.g., through averaging. Discussion of deciding how many replication to run, while very important, is beyond the scope of this primer.



[Chapter 10] **Fundamentals of Output Analysis**

Time-series plot by type

The second charting example is also a system-level time-series chart that shows the contents of the ContainerStorage queue by product type, as opposed to the total of all products. The chart is shown in the figure to the right. It again clearly depicts the dynamic and stochastic nature of the system.



This chart is created in a manner similar to the Content chart above.

- Select the **WIP By Type** chart type in the **Content** templates section of the Dashboard Library; then, select **Line Chart** from the pop-up menu.
- Drag the chart Library item onto the Dashboard panel where it can be sized and positioned.
- Rename the chart to something meaningful, for the example, **Contents of Container Storage by Type**.

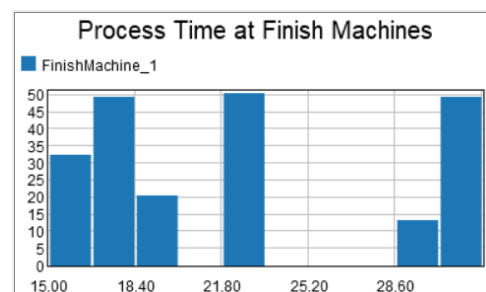
- On the Options tab, click the + button in the **Entrance Objects** section and select **Select Objects** from the drop-down menu. Again, this generates a list of all of the objects that are currently in the model by category.
- From the **Queues** category, select the object ContainerStorage so that it is highlighted and press the **Select** button. The ContainerStorage object is now displayed in the **Entrance Objects** section.
- Similarly, again on the Options tab, click the + button in the **Exit Objects** section and select **Select Objects** from the drop-down menu. From the **Queues** category, select the object ContainerStorage so that it is highlighted and press the **Select** button. The ContainerStorage object is now displayed in the **Exit Objects** section.
- As with the Content chart, the only other parameter that should be changed for now is the **Time Axis Mode** property on the Settings tab. Change the value from the default **Show exact time and date** to **Show Duration**. Also, change the displayed units from **Seconds** to **Hours**.
- Since only there are multiple values being plotted on the one chart, leave the **Show Legend** box checked (again, on the Settings tab).
- Relocate the chart on the Dashboard.
- The default settings for all of the other chart parameters are fine for now. Therefore, press OK to close the interface.

Note that the chart could track the contents of multiple objects over time; e.g., to track the number of items in the queue and the processor, then the Exit Objects would be changed to the Processor, FinishMachine_1.

Also note that the type can be any label value; this example used the Type label, which is quite common. Type is defined at the bottom of the Options tab.

Histogram plot

The third charting example is also a system-level chart and, as shown in the figure to the right, it considers the time an item spends in the Processor (at the finish machine). The stay time includes the three constant process times (15, 20, and 30 minutes) and the 2-minute setup times when they are needed in the operation, i.e. when the item type changes.



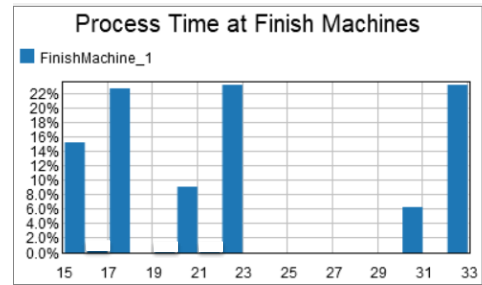
This chart is created in a manner similar to the Content charts described above.

- Select the **Staytime** chart type in the **Staytime** templates section of the Dashboard Library; then select **Histogram** from the pop-up menu.
- Drag the chart Library item onto the Dashboard panel where it can be sized and positioned.
- Rename the chart to something meaningful, for the example, **Process Times at Finish Machines**.
- The only required input is the selection of which object(s) is the subject of the plot. As before, click the + in the **Objects** section of the Options tab., then select the **Select Objects** option.
- Select FinishMachine_1 from the **Processor** section and then press the **Select** button at the bottom of the interface.

The resulting chart, when run for 4800 minutes, should look similar to the one in the figure above. While useful, the histogram can be reformatted to be more readable. Of course, the underlying data are not changed, just the way the data are summarized in the chart.

The following two additional steps make the chart correspond to the one to the right. The x and y axes are customized for readability and interpretation.

- Change the scale of the y-axis to display the frequency of the histogram as percentages (relative values) rather than the default, absolute number of occurrences. To do this, check the box **Normalize Values** on the Settings tab.
- Change the definition of the bars, the x axis, from a fixed number of bars (default is 10) to a fixed bucket width. To do this, again on the Settings tab, change the **Bar Mode** from **By Number of Buckets** to **By Bucket Width**. For this example, set the Bucket Width from the default of 50.0 to 1 (1 minute) and the Bucket Offset, the starting point of chart, from the default of 0.0 to 15 (the shortest processing time is 15 minutes with no setup).



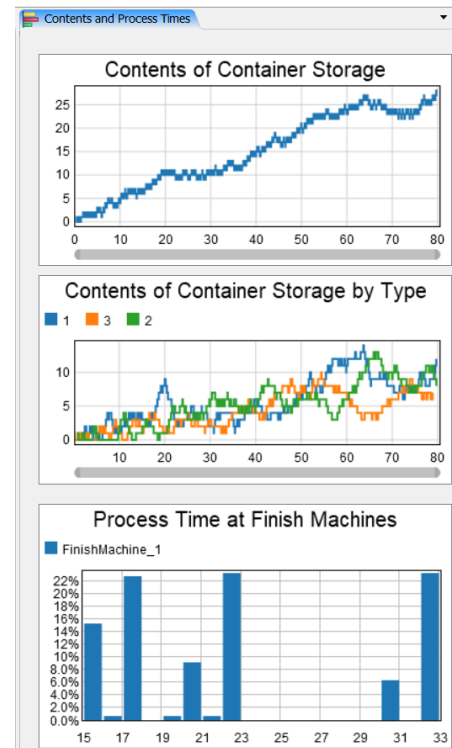
Setting the number of bars and bar width for histograms are oftentimes done by trial and error. However, there are some methodologies to help with this; three such methodologies are available in *FlexSim* – Sturges', Scott's, and Freedman-Diaconis Rules. They are available when selecting **By Bucket Width** property. Discussion of these methodologies is beyond the scope of the primer.

- On the Text tab, change the **Precision** property from the default of 2 (2 decimal places) to 0.
- Press **OK** to close the chart window.

The final dashboard should resemble the figure to the right.

The dashboard has been renamed **Contents and Process Times** since other dashboards will be created later and the descriptive name helps for reference.

- Change the name in the Dashboard Name field in the Quick Properties window, as shown in the figure to the right.





Charts provide a very valuable means for verifying and validating simulation models.

Note the following in the charts on the Dashboard above.

- As shown in the first two plots, the contents of the container storage is growing over time. This is an obvious indicator that the system is unsustainable and not feasible, since in practice queues cannot continue to grow in size. It was indicated earlier that one solution would be to add another finish machine, which will be done later in the primer.
- The third plot, the histogram of staytimes, verifies the three deterministic process times by container type (15, 20, and 30 minutes), with approximate frequencies of occurrence of 15%, 10%, and 7%, respectively. The values of 17, 22, and 32 are the process times by type including setup times; their frequencies are approximately 22%, 23%, and 23%, respectively. Obviously, the setup operation is required more often than not.

Also note that the total percentage of Type 1 is 37% (15%+22%), Type 2 is 33% (10% + 23%), and Type 3 is 30%. (7% + 23%). This is close to the assumption that the products are equally likely to occur. Of course, they will not be exactly equal to 33.33% for any run due to sampling error. However, over the long run, the product mix should be equally likely.



If you haven't already done so, save the model. Recall, it is good practice to save often.

This page is intentionally blank.